

**Haute Ecole LÉONARD DE VINCI
INSTITUT PAUL LAMBIN**

Section Informatique

Université Catholique de Louvain

**Design et implémentation d'une
suite d'outils capable de monter
en charge, flexible et sécurisée
pour l'évaluation automatique
de programmes d'étudiants**

Mémoire présenté par

CHALET, LIONEL

en vue de l'obtention du diplôme de
bachelier en Informatique de gestion

Année académique 2013-2014

Remerciements

Je tiens à remercier tout particulièrement José Vander Meulen qui m'a permis de trouver mon stage.

Je remercie mon promoteur Pierre Schaus de m'avoir donné du travail intéressant, d'avoir suivi l'évolution de celui-ci et de m'avoir conseillé sur les outils à utiliser. Je le remercie aussi pour sa lecture de ce mémoire.

Ensuite, j'exprime ma gratitude à Stéphanie Ferneew pour sa relecture attentive de ce mémoire ainsi que pour sa réactivité par e-mails.

Je tiens également à remercier Antoine et Simon, avec qui j'ai partagé un bureau durant la quasi totalité de mon stage, pour avoir égayé mon séjour à l'UCL.

Comment ne pas remercier Vanessa pour son beau sourire et la présence féminine qu'elle apporte au département.

Je souhaite enfin exprimer ma gratitude aux professeurs de l'IPL qui m'ont appris beaucoup de choses ces trois dernières années.

Table Des Matières

Préambule	1
1. Introduction	1
2. Contexte de travail	2
a. L'UCL et ses départements	2
b. L'environnement informatique	2
c. Ma place dans l'équipe	2
3. Calendrier	3
Application	5
1. Mise en situation	5
2. Notions préliminaires	6
a. Logiciel de gestion de versions	6
1. Définition	6
2. Logiciel de gestion de versions centralisé	6
3. Logiciel de gestion de versions décentralisé	7
4. Les hooks	8
b. SSH	9
3. Analyse	10
a. Diagramme des cas d'utilisation	10
b. Modèles complets	11
1. Cas d'utilisation « Créer une soumission »	11
a. Modèle	11
b. Scénario nominal	12
c. Scénarios alternatifs	14
2. Cas d'utilisation « Participer à une soumission »	15
a. Modèle	15
b. Scénario nominal	16
c. Scénarios alternatifs	16
3. Cas d'utilisation « Soumettre une solution »	17
a. Modèle	17
b. Scénario nominal	18
4. Outils utilisés	19
a. Parallélisation de traitement	19
b. Sandboxing	21
1. Problématique	21
2. Contrôle d'accès obligatoire avec SELinux et AppArmor	21
3. Virtualisation	21
a. Différentes techniques	21
b. Noyau en espace utilisateur avec UML	21
c. Hyperviseur de type 1 avec Xen et KVM	22
d. Hyperviseur de type 2 avec VMware et Vagrant	22
e. Isolateur avec OpenVZ et LXC	23
5. Frameworks de tests	25
a. Description	25
b. Frameworks développés	25
1. Java	25
2. Bash	27
6. Gestion des soumissions	28
a. Serveur Apache2	28
1. Introduction	28
2. Django	28
a. Introduction	28
b. Schéma entité-relation	29
3. Flask et Google+ API	30
a. Introduction	30
b. Google+ Sign-In	30
4. jQuery	32
5. Mise en page avec Twitter Bootstrap	32
b. Création	33
1. Introduction	33
2. Création d'un repository	33

3. Création d'un accès SSH	34
4. Création d'un Google Apps Script	34
5. Déploiement en application web	36
6. Association de l'application web au repository	36
c. Modification	37
d. Participation	37
1. Introduction	37
2. Création d'un subrepository	37
3. Création d'un accès SSH	37
7. Présentation des résultats	38
a. E-mail	38
b. Application web Google	38
1. Introduction	38
2. Contenu générique	38
a. Google Apps Script	38
b. jQuery et Twitter Bootstrap	39
3. Contenu spécifique	40
a. Description	40
b. Exemples	41
4. Contenu généré	42
c. Mercurial	43
Conclusion	44
Glossaire	46
Bibliographie	48
Annexes	49

Table Des Figures

Figure 1 : logiciel de gestion de versions centralisé	6
Figure 2 : logiciel de gestion de versions décentralisé	7
Figure 3 : diagramme des cas d'utilisation	10
Figure 4 : virtualisation avec noyau en espace utilisateur	21
Figure 5 : virtualisation avec hyperviseur de type 1	22
Figure 6 : virtualisation avec hyperviseur de type 2	22
Figure 7 : virtualisation avec isolateur	23
Figure 8 : schéma entité-relation de l'application Django	29
Figure 9 : OAuth : one-time code flow	31
Figure 10 : application web de Scavenbox	32
Figure 11 : OAuth : service account flow	35
Figure 12 : déploiement d'un Google Apps Script en application web	36
Figure 13 : Google Apps Script déployé en application web	40
Figure 14 : bouton de soumission sur un GAS pour le type "stand-alone"	40
Figure 15 : exemple de fichier contenant des besoins et des coûts de production	42
Figure 16 : fichier de configuration de Mercurial	43

Table Des Codes Sources

Code 1 : définir un hook sur un push dans Mercurial	8
Code 2 : exemple de programme appelé par un hook	8
Code 3 : descripteur de tâche Condor	20
Code 4 : mécanisme de ClassAd de Condor	20
Code 5 : édition du fichier /etc/sudoers avec visudo	24
Code 6 : exemple de rapport de tests du framework de tests Java	26
Code 7 : commentaires pour donner un temps d'exécution et un score à un test pour le framework de tests Bash	27
Code 8 : exemple de rapport de tests du framework de tests Bash	27
Code 9 : exemple de formulaire avec Django	28
Code 10 : exemple de modèle avec Django	29
Code 11 : obtention des droits à utiliser l'API Google Drive	35
Code 12 : fonction doGet d'un Google Apps Script	39

Préambule

1. Introduction

DESCRIPTION DU SUJET

Afin de conclure mes études à l'Institut Paul Lambin, j'ai eu l'opportunité d'effectuer un stage de quatorze semaines. Les notions enseignées à l'institut m'ont permis d'aller plus loin et de découvrir davantage ce monde merveilleux qu'est l'informatique.

J'ai effectué ce stage à l'Université Catholique de Louvain, au pôle d'ingénierie informatique. Les instructeurs de ce pôle dispensent des cours d'informatique et souhaitent avoir un feedback automatique sur le code que les étudiants leur soumettent. Ce mémoire traite de la solution que j'ai conçue et implémentée durant mon stage pour répondre à ce besoin. Elle porte le doux nom de *Scavenbox*.

Le feedback est donné par l'exécution d'une batterie de tests qui produit un résultat. Bien sûr, il est impossible de faire des tests qui corrigent n'importe quel code. Chaque instructeur souhaitant que ses étudiants résolvent un problème devra donc écrire une batterie de tests spécifiques à ce problème.

OBJECTIFS DU PROJET

La solution développée se devant d'être facile à maintenir, le maître mot sera de ne pas réinventer la roue. Aussi, plutôt qu'un développement fastidieux d'une solution complète ne se basant sur aucun outil existant, nous préferons assembler intelligemment des outils existants et éprouvés. Nous aurons ainsi un nombre de lignes de code limité à maintenir.

Scavenbox doit permettre de lancer les tests au simple clic d'un bouton ou bien à la simple exécution d'une commande. Nous ne voulons en aucun cas que cela nécessite des manipulations complexes comme la copie du code source.

Bien qu'étant développé pour l'UCL dans un premier temps, Scavenbox se devra d'être installable sur n'importe quels serveurs et de pouvoir monter en charge au besoin, c'est-à-dire qu'il doit être possible de paralléliser le traitement des tests sur une grappe de serveur.

Un certain nombre de langages de programmation sont utilisés à l'UCL, mais pas tous. Il faut donc permettre l'usage de Scavenbox pour n'importe quel langage de programmation en permettant de lui intégrer des frameworks¹ de tests supplémentaires.

Gardons à l'esprit que du code d'étudiant sera exécuté sur nos serveurs. Nous ignorons, à priori, ce que ce code fait mais nous savons qu'il est potentiellement mal intentionné. Dans un souci de sécurité nous voulons donc que les tests soient exécutés dans un environnement cloisonné. Il doit être impossible de mettre les serveurs sur lesquels Scavenbox tourne par terre.

Les outils utilisés par Scavenbox doivent être des logiciels libres, c'est à dire des logiciels dont l'utilisation et la modification sont autorisés pour tous les types d'utilisation. Cette notion de logiciel libre est à opposer aux logiciels propriétaires qui ne permettent pas d'être exécutés pour tout type d'utilisation.

Je vous souhaite une agréable lecture.

¹ *Framework*, voir glossaire page 46

2. Contexte de travail

a. L'UCL et ses départements

L'Université catholique de Louvain fut fondée à Malines en 1834. L'année qui suivit, elle fut installée à Louvain. Cette université avait pour ambition de faire renaître la gloire de l'ancienne Université de Louvain, fondée en 1425 et dissoute en 1797. Entre les deux il eut l'Université d'Etat de Louvain de 1817 à 1831.

L'UCL francophone et basée à Louvain-la-Neuve telle que nous la connaissons aujourd'hui ne fit son apparition qu'en 1968, suite à la scission de l'université en deux entités indépendantes. L'UCL compte aujourd'hui près de 30 000 étudiants et propose un large choix de formations.

J'ai travaillé au sein de l'Ecole polytechnique de Louvain (EPL) qui offre le choix entre les formations d'ingénieur civil et d'informaticien. L'ICTEAM (Institute of Information and Communication Technologies, Electronics and Applied Mathematics) est un institut faisant partie de l'EPL. L'un des pôles de cet institut est le pôle INGI, le pôle d'ingénierie informatique, au sein duquel j'ai effectué mon stage.

b. L'environnement informatique

Le pôle INGI dispose de salles informatiques. Condor² est déployé sur chaque machine pour un total de 90 nœuds d'exécution.

Il existe également des serveurs qui font tourner Pythia, le logiciel que Scavenbox est amené à remplacer. Scavenbox devrait à terme tourner sur ces serveurs.

c. Ma place dans l'équipe

Mon promoteur, Pierre Schaus, m'a chargé de développer une alternative à Pythia. J'étais seul sur ce nouveau projet et je n'ai donc pas eu l'occasion de travailler en équipe. J'étais par ailleurs seul dans mon bureau les premières semaines. Cela n'étant pas très vivant j'ai ensuite déménagé dans un autre bureau où j'ai eu l'occasion de côtoyer 2 chercheurs, Antoine et Simon.



² Voir page 19

3. Calendrier

SEMAINE 1

Du 24 AU 28 FÉVRIER

- Découverte de Mercurial et de ses hooks³ (*voir page 8*)
- Implémentation d'un framework de tests Java consistant en une surcouche de JUnit pour permettre de pondérer les tests (*voir page 25*)
- Découverte de Condor (*voir page 19*)

SEMAINE 2

Du 3 AU 7 MARS

- Configuration de Condor
- Découverte de User Mode Linux (UML) (*voir page 21*)
- Problèmes pour faire tourner UML dans Condor

SEMAINE 3

Du 10 AU 14 MARS

- Développement d'une application web en Python avec Django sur l'exemple du problème du voyageur de commerce (*voir page 28*)

SEMAINE 4

Du 17 AU 21 MARS

- Abandon de l'application web Django
- Développement d'un équivalent avec Google Apps Script⁴ (GAS), déployable en application web (*voir page 38*)
- Enjoliment de l'application web avec Twitter Bootstrap (*voir page 39*)

SEMAINE 5

Du 24 AU 28 MARS

- Retour sur les problèmes de compatibilité de UML et Condor, abandon de UML
- Recherche infructueuse d'une alternative à UML : Xen, KVM, Vagrant, VMware, SELinux, AppArmor, OpenVZ (*voir page 22*)
- Découverte de LXC (Linux Containers), mon sauveur (*voir page 23*)

SEMAINE 6

Du 31 MARS AU 4 AVRIL

- Implémentation du lien Mercurial -> Condor -> LXC -> exécution de tests
- Découverte du Google Drive SDK et implémentation d'un programme en Python qui exporte des GAS sur Google Drive (*voir page 34*)

³ Hook, voir glossaire page 47

⁴ Google Apps Script, voir glossaire page 46



SEMAINE 7

DU 7 AU 11 AVRIL

- Ecriture de scripts Bash pour la gestion des soumissions⁵
- Création d'accès restreints à certains repositories⁶ en fonction d'une clé SSH avec hg-gateway (*voir page 34*)
- Installation d'un serveur Apache2 (*voir page 28*)
- Découverte de la Google+ API (*voir page 30*)

SEMAINE 8

DU 14 AU 18 AVRIL

- Implémentation d'une application web en Python avec Flask pour se connecter avec son compte Google et appeler les scripts de gestion des soumissions (*voir page 30*)

SEMAINE 9

DU 21 AU 25 AVRIL

- Suite de l'application web + utilisation de jQuery pour faire des requêtes Ajax⁷ et de Twitter Bootstrap pour le CSS (*voir page 32*)

SEMAINE 10

DU 28 AVRIL AU 2 MAI

- Catégorisation des soumissions en 3 types (*voir page 33*)
- Ajout de la possibilité d'ajouter des frameworks de tests pour d'autres langages (le seul framework développé jusqu'ici est pour le langage Java) (*voir page 25*)
- Implémentation d'un changement de propriétaire après exportation des GAS sur Google Drive + implémentation d'un GAS pour contourner un bug de Google lors du déploiement d'un GAS en application web après changement de propriétaire (*voir page 36*)

SEMAINE 11

DU 5 AU 9 MAI

- Correction de bugs en tout genre
- Présentation de la solution au département + collecte de feedback
- Utilisation des subrepositories⁸ Mercurial selon les remarques du département (*voir page 37*)

SEMAINE 12

DU 12 AU 16 MAI

- Amélioration de la présentation des résultats (*voir page 43*)
- Ecriture d'un framework de test en Bash (*voir page 27*)

SEMAINE 13

DU 19 AU 23 MAI

- Tests fonctionnels de l'application et correction de bugs
- Ecriture de commentaires pour faciliter la lecture du code
- Ecriture de la documentation

SEMAINE 14

DU 26 AU 30 MAI

- Finalisation du mémoire

⁵ *Soumission*, voir glossaire page 47

⁶ *Repository*, voir glossaire page 47

⁷ *Ajax*, voir glossaire page 46

⁸ *Subrepository*, voir glossaire page 47

Application

1. Mise en situation

A mon arrivée, le pôle INGI bénéficiait du logiciel Pythia. Pythia⁹ est un logiciel qui permet déjà de soumettre du code d'étudiant à une batterie de test. Mais ce logiciel présente certains inconvénients. En effet, il n'est pas terminé. Il faudrait donc pouvoir continuer son développement et ensuite le maintenir. Mais Pythia ayant été développé de toutes pièces sans faire beaucoup appel à des outils existants, son code est d'une longueur conséquente et personne en INGI n'a ni l'envie ni le temps de se plonger dedans. Pythia est également développé en grande partie en Go, un nouveau langage de programmation introduit par Google. Bien que n'étant pas un mauvais langage, il est inconnu au bataillon et augmente donc encore la difficulté à le maintenir. De plus l'interface utilisateur pour lancer les tests n'est pas des plus pratiques puisqu'il s'agit de copier/coller le code sur un site avant de cliquer sur un bouton pour lancer les tests. Autant cela n'est pas trop dérangeant si le code tient en un seul fichier, autant cela devient problématique dès qu'il y en a plusieurs. Scavenbox se veut donc de palier aux défauts de Pythia. Tout d'abord en utilisant des outils existants et qui ont fait leurs preuves. Ensuite en simplifiant la procédure de soumission de code et la récupération des résultats.

Scavenbox a été pensé dès le départ pour être utilisé sur des MOOC, ou Massive Open Online Course, c'est-à-dire un cours en ligne massif. Nous retrouvons ici l'objectif de capacité à monter en charge de l'application. Certains MOOC ont des cours qui accueillent jusqu'à 100 000 participants et il est donc impératif d'utiliser un logiciel de correction automatique pour vérifier des solutions de participants au cours.

Un exemple de MOOC très connu est Coursera¹⁰, qui permet de suivre des cours en ligne massifs et ce de manière gratuite.

Scavenbox est prévu pour tourner sur une distribution Debian ou bien sur une distribution dérivée telle que Ubuntu. Il faut y créer un utilisateur *submit* disposant d'un répertoire home.



⁹ Le site de Pythia est disponible à l'adresse suivante : <http://pythia.info.ucl.ac.be/home>

¹⁰ Le site de Coursera est disponible à l'adresse suivante : <https://www.coursera.org/>

2. Notions préliminaires

a. Logiciel de gestion de versions

1. Définition

Nous souhaitons que le déclenchement de la procédure pour soumettre une solution soit la plus simple possible, et ce même si le code à tester est formé de plusieurs fichiers. C'est ici qu'apparaît la notion de logiciel de gestion de versions. Un logiciel de gestion de versions permet d'enregistrer des versions d'un projet informatique. Si l'étudiant possède une version du projet et qu'un serveur en détient une autre plus ancienne, l'étudiant peut mettre à jour la version du serveur. Le serveur peut alors détecter le changement et lancer des tests sur la nouvelle version du projet.

On parle d'un repository, ou d'un dépôt en français, pour désigner le dossier dont on veut garder l'historique des versions. On peut enregistrer une version, on parle dès lors d'un *commit*. On peut également charger une version, on parle dès lors d'un *update*. Par défaut, un update charge la dernière version du projet mais on peut également choisir de revenir à une version plus ancienne.

Une des forces des logiciels de gestion de versions est qu'ils offrent la possibilité de travailler à plusieurs sur le même projet. Il existe pour cela des logiciels de gestion de versions centralisés ou décentralisés.

2. Logiciel de gestion de versions centralisé

Un logiciel de gestion de versions centralisé, comme Subversion par exemple, interagit toujours avec le même serveur. Ainsi, si 2 étudiants veulent travailler sur le même projet, ils sont obligés de passer par le serveur pour faire leurs commits.

Hors, dans notre idée, nous souhaitons que chaque enregistrement de version sur le serveur lance l'exécution de tests. Comme nous pouvons le constater sur la *Figure 1*, un étudiant A qui souhaite partager son avancement sur le projet à un étudiant B doit faire un commit sur le serveur. Ce commit lance l'exécution de tests alors que le projet n'est peut-être pas terminé et qu'il n'était peut-être pas dans l'intention de A de déjà le tester.

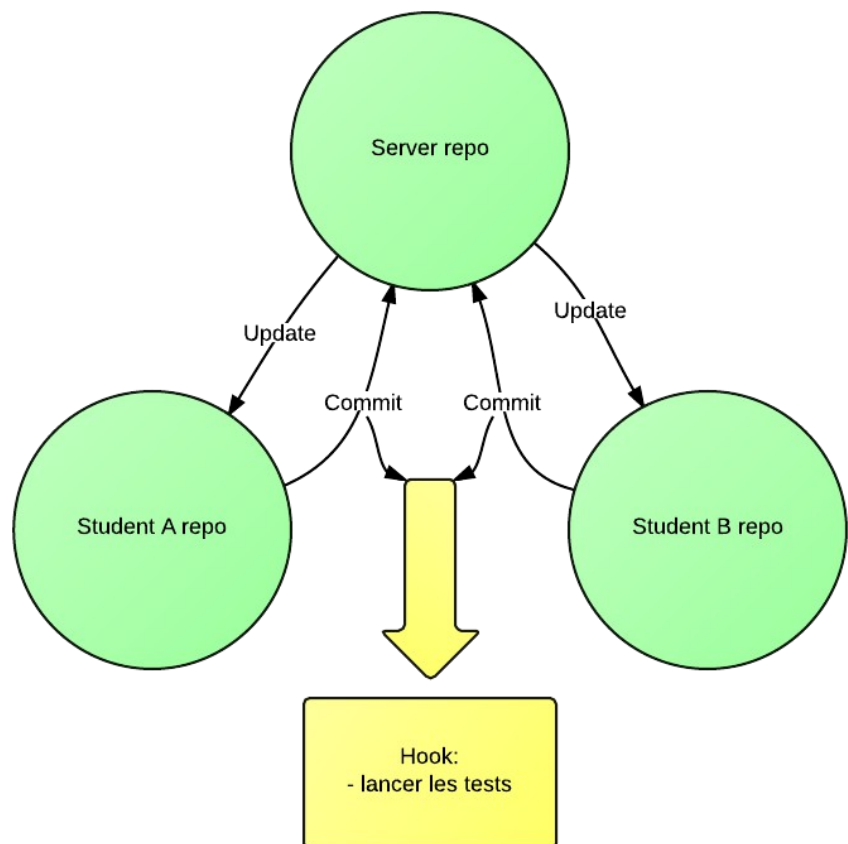


Figure 1 : logiciel de gestion de versions centralisé

NOTIONS PRÉLIMINAIRES

Cette solution ne nous convient donc pas et il nous faut trouver un mécanisme qui permette d'échanger des versions sans passer par le serveur.

3. Logiciel de gestion de versions décentralisé

Un logiciel de gestion de versions décentralisé permet de faire cela, comme on peut le voir sur la *Figure 2*. En effet, on ne doit plus obligatoirement passer par le serveur pour s'échanger des versions. Les updates et les commits se font en local contrairement à un logiciel de gestion de versions centralisé. Un partage de version se fait avec un *push* ou un *pull*. Un push donne une de nos versions à un collaborateur du projet tandis qu'un pull nous permet de prendre une de ses versions.

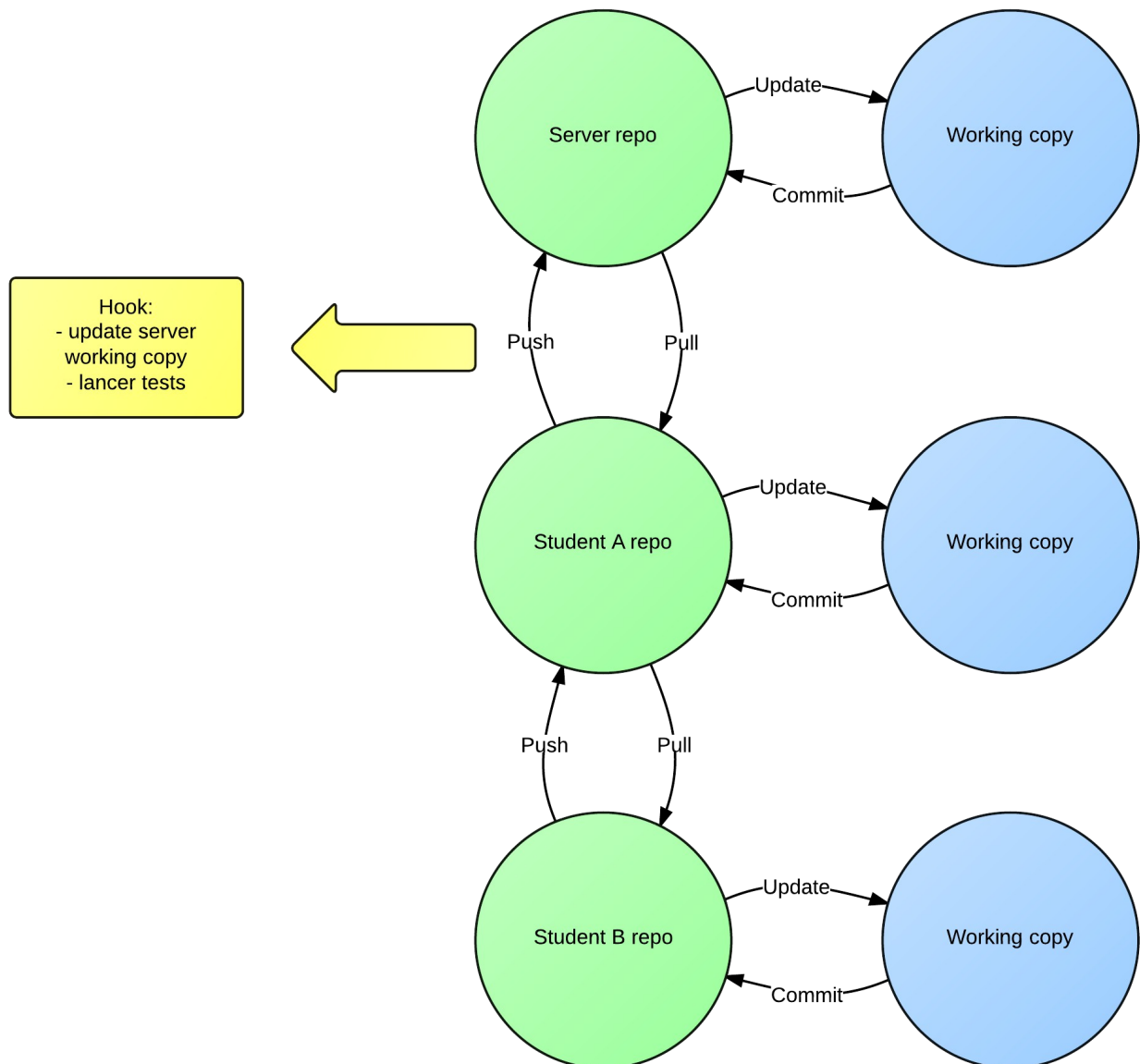


Figure 2 : logiciel de gestion de versions décentralisé

NOTIONS PRÉLIMINAIRES

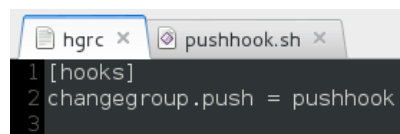
Constatons que l'on peut donc échanger des versions sans passer par le serveur. Une fois que les étudiants ont atteint une version qu'ils estiment testable, ils peuvent l'envoyer au serveur qui démarrera alors l'exécution des tests.

Il existe deux logiciels de gestions de versions renommés : Git et Mercurial. L'un ou l'autre nous aurait tout autant convenu, mais nous avons choisi d'utiliser Mercurial pour des raisons d'habitudes au sein du pôle INGI.

4. Les hooks

L'exécution des tests sur le serveur lors d'un push ne se fait pas automatiquement. Il faut donc trouver une manière de déclencher cette exécution. C'est une fonctionnalité prévue dans Mercurial (aussi bien que dans Git) et qui est assez facile d'utilisation.

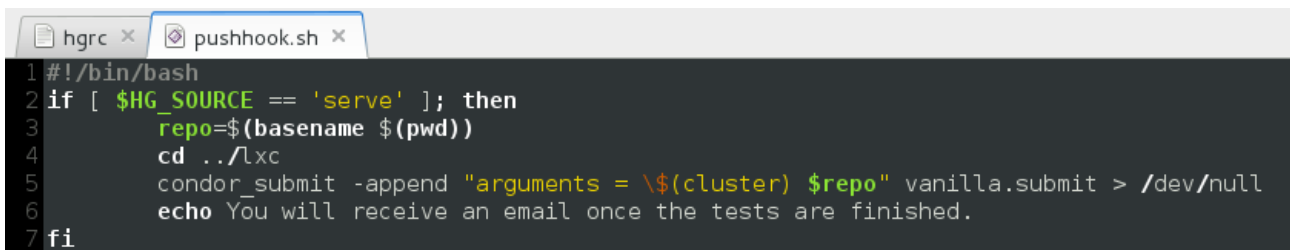
Un repository Mercurial contient un dossier nommé `.hg` dans lequel les informations du repository sont stockées. On peut dans ce dossier créer un fichier nommé `hgrc` qui contiendra des options de configuration du repository. Une de ces options est de définir un hook comme nous le montre le *Code 1*.



```
1 [hooks]
2 changegroup.push = pushhook
3
```

Code 1 : définir un hook sur un push dans Mercurial

Ce petit bout de code indique que le programme `pushhook` sera exécuté à chaque push dans le repository concerné. Le code de ce programme est très simple (voir *Code 2*).



```
1 #!/bin/bash
2 if [ $HG_SOURCE == 'serve' ]; then
3     repo=$(basename $(pwd))
4     cd ../lxc
5     condor_submit -append "arguments = \$(cluster) $repo" vanilla.submit > /dev/null
6     echo You will receive an email once the tests are finished.
7 fi
```

Code 2 : exemple de programme appelé par un hook

Ce programme est en fait un script bash qui se charge de démarrer l'exécution des tests. La ligne de code intéressante la dedans est la ligne n°5, celle avec la commande `condor_submit`. Son action est de soumettre un job à Condor, le programme de parallélisation que nous utilisons pour exécuter les tests.¹¹

¹¹ Voir page 19

b. SSH

Nous parlerons à plusieurs reprises dans ce document de SSH (Secure Shell). SSH est un protocole de communication sécurisé qui chiffre tous les segments TCP envoyés. Mais avant de pouvoir communiquer entre elles, 2 machines doivent se connecter l'une à l'autre. Cela peut se faire par mot de passe mais on préfère en général procéder à l'aide de clés de chiffrement asymétriques. Il existe des clés publiques et des clés privées, elles fonctionnent toujours par paire.

Imaginons qu'Alice ait envie de recevoir des messages de Bob et qu'elle dispose d'une paire de clés, une privée et une publique. Alice garde sa clé privée et donne sa clé publique à Bob. Bob peut maintenant encrypter des messages avec la clé publique et les envoyer à Alice qui les décryptera à l'aide de sa clé privée. C'est le fonctionnement de base de la cryptographie asymétrique. Mais dans le cas d'une connexion SSH, ce qu'Alice souhaite faire n'est pas seulement recevoir des messages de Bob mais tout d'abord lui certifier qu'elle est bien Alice. Pour ce faire, elle envoie un message signé avec sa clé privée. Bob ne pourra lire ce message avec sa clé publique que s'il a été encrypté par la seule clé privée correspondante, c'est-à-dire la clé privée d'Alice. Cela certifie donc son identité.



3. Analyse

a. Diagramme des cas d'utilisation

Scavenbox est capable de mener à bien certains cas d'utilisation. Les acteurs de ces cas d'utilisation sont soit des instructeurs, soit des étudiants. Un instructeur peut être un professeur ou un assistant.

Voici le diagramme des cas d'utilisation de Scavenbox :

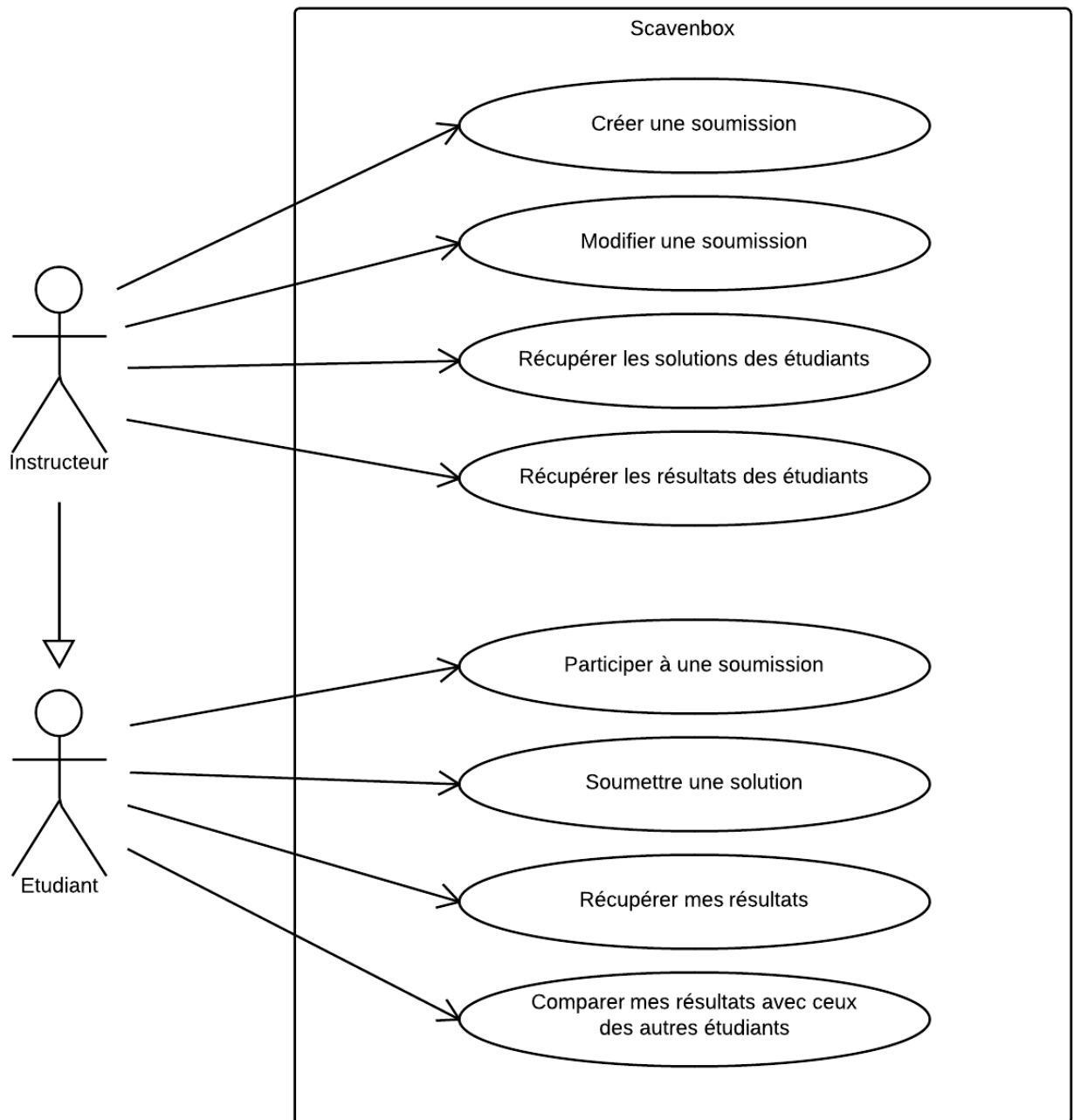


Figure 3 : diagramme des cas d'utilisation

Constatons que l'acteur *Instructeur* peut faire tout ce que l'acteur *Etudiant* peut faire. Le serveur sait qu'il s'agit d'un instructeur et pas d'un étudiant lorsque l'adresse e-mail dudit instructeur se trouve dans un fichier spécifique sur le serveur. (le fichier *~/profs*)

Notons la différence entre les cas d'utilisation *Récupérer les solutions des étudiants* et *Récupérer les résultats des étudiants*. Pour la solution, il s'agit de récupérer le code que l'étudiant a soumis. Pour son résultat, il s'agit de récupérer le résultat que sa solution a obtenu aux tests de la soumission.

Les cas d'utilisation les plus intéressants sont *Créer une soumission*, *Participer à une soumission* et *Soumettre une solution*. Les modèles complets de ces trois cas d'utilisation sont détaillés ci-après.

Le cas d'utilisation *Créer une soumission* est détaillé à la rubrique *Création* page 33. Le cas d'utilisation *Modifier une soumission* est expliqué à la rubrique *Modification* page 37. Plus de détails sur le cas d'utilisation *Participer à une soumission* sont disponibles à la rubrique *Participation* page 37. Le cas d'utilisation *Soumettre une solution* est abordé au chapitre *Outils utilisés* page 19. Le cas d'utilisation *Comparer mes résultats avec ceux des autres étudiants* est détaillé à la rubrique *Application web Google* page 38. Les cas d'utilisation *Récupérer les solutions des étudiants*, *Récupérer les résultats des étudiants* et *Récupérer mes résultats* sont expliqués à la rubrique *Mercurial* page 43.

b. Modèles complets

1. Cas d'utilisation « Créer une soumission »

a. Modèle

Le cas d'utilisation "Créer une soumission" permet à un instructeur de créer une soumission à laquelle ses étudiants peuvent participer. Il doit pour cela fournir le code source des tests qu'il a écrit pour tester un projet qu'il a donné à ses étudiants.

Nom du cas d'utilisation	Créer une soumission
Objectif	Créer une soumission
Niveau	Tâche utilisateur
Acteur principal	Instructeur
Parties prenantes et intérêts	
Pré-conditions	L'instructeur est connecté et le système connaît son nom, son prénom et son adresse e-mail
Etat après réussite	La soumission est créée dans le système et les étudiants peuvent y participer
Etat après échec	La soumission n'est pas créée et les étudiants ne peuvent pas y participer

Trigger	L'instructeur déclenche la procédure pour créer une soumission
Scénario nominal	Voir page 12
Alternatives	Voir page 14

Les notes de bas de page présentes sous le scénario nominal indiquent où aller dans le présent document pour avoir plus d'informations sur la mise en œuvre de la ligne concernée. De manière générale, vous trouverez plus d'informations sur ce cas d'utilisation à la rubrique *Création*, page 33.

b. Scénario nominal

UTILISATEUR	SYSTÈME
1. L'utilisateur déclenche la procédure pour créer une soumission	
2. L'utilisateur donne un nom de soumission	
3. L'utilisateur sélectionne une archive contenant les fichiers de tests de la soumission	
4. L'utilisateur donne une clé SSH publique	
	5. Le système vérifie que le nom de la soumission n'est pas déjà pris
	6. Le système vérifie que l'archive est valide
	7. Le système vérifie que la clé SSH publique est valide
	8. Le système exporte sur son Google Drive les fichiers nécessaires à la création d'une application web ¹²
	9. Le système définit l'utilisateur comme étant le nouveau propriétaire des fichiers exportés au point précédent
	10. Le système crée un repository sur le serveur ¹³
	11. Le système limite l'accès au repository créé au point précédent avec la clé SSH publique donnée au point 4 ¹⁴

¹² Voir *Création d'un Google Apps Script* page 34

¹³ Voir *Création d'un repository* page 33

¹⁴ Voir *Création d'un accès SSH* page 34

	12. Le système envoie un e-mail à l'utilisateur contenant un lien et un mot de passe
13. L'utilisateur clique sur le lien dans l'e-mail qu'il a reçu	
	14. Le système crée une copie d'un des fichiers exportés au point 8 : le Google Apps Script ¹⁵
	15. Le système présente un lien vers le fichier copié au point précédent
16. L'utilisateur clique sur le lien dans l'e-mail	
17. L'utilisateur autorise l'exécution du Google Apps Script	
18. L'utilisateur enregistre une version du Google Apps Script et le déploie en application web	
19. L'utilisateur redonne le nom de la soumission et donne le mot de passe reçu au point 12	
20. L'utilisateur donne l'identifiant de l'application web Google déployée au point 18	
	21. Le système associe l'identifiant donné au point précédent au repository créé au point 10 ¹⁶
	22. Le système envoie un e-mail à l'utilisateur

15 Voir *Déploiement en application web* page 36, la copie est nécessaire à cause d'un bug sur Google Drive

16 Voir *Association de l'application web au repository* page 36

c. Scénarios alternatifs**4.A. [L'UTILISATEUR NE DONNE PAS DE CLÉ SSH]**

- 4.A.1. Le système vérifie que l'archive est valide
- 4.A.2. Branchement au point 8

5.A. [LE NOM DE LA SOUMISSION EST DÉJÀ PRIS]

- 5.A.1. Branchement au point 2

6.A. [L'ARCHIVE N'EST PAS VALIDE]

- 6.A.1. Branchement au point 2

7.A. [LA CLÉ SSH PUBLIQUE N'EST PAS VALIDE]

- 7.A.1. Branchement au point 2

8.A. [L'EXPORTATION DES FICHIERS SUR LE GOOGLE DRIVE A ÉCHOUÉ]

- 8.A.1. Branchement au point 2

10.A. [L'UTILISATEUR N'A DONNÉ AUCUNE CLÉ SSH AU POINT 4]

- 10.A.1. Le système envoie un e-mail à l'utilisateur contenant un lien
- 10.A.2. Branchement au point 13

19.A. [L'UTILISATEUR N'A DONNÉ AUCUNE CLÉ SSH AU POINT 4]

- 19.A.1. Réussite du cas d'utilisation

2. Cas d'utilisation « Participer à une soumission »

a. Modèle

Le cas d'utilisation "Participer à une soumission" permet à un étudiant de participer à une soumission créée par un de ses instructeurs. Il pourra ensuite soumettre ses solutions et lancer l'exécution de tests dessus.

Nom du cas d'utilisation	Participer à une soumission
Objectif	Participer à une soumission
Niveau	Tâche utilisateur
Acteur principal	Etudiant
Parties prenantes et intérêts	
Pré-conditions	La soumission a été créée par un instructeur, le mot passe de la soumission a été transmis à l'étudiant par l'intermédiaire de l'instructeur, l'étudiant est connecté et le système connaît son nom, son prénom et son adresse e-mail
Etat après réussite	L'étudiant est enregistré comme participant à la soumission et il peut soumettre des solutions
Etat après échec	L'étudiant n'est pas enregistré comme participant à la soumission et il ne peut pas soumettre de solutions
Trigger	L'étudiant déclenche la procédure pour participer à une soumission
Scénario nominal	Voir page 16
Alternatives	Voir page 16

Comme indiqué dans les pré-conditions, il relève de la responsabilité de l'instructeur de transmettre le mot de passe d'une soumission à ses étudiants.

Les notes de bas de page présentes sous le scénario nominal indiquent où aller dans le présent document pour avoir plus d'informations sur la mise en œuvre de la ligne concernée. De manière générale, vous trouverez plus d'informations sur ce cas d'utilisation à la rubrique *Participation*, page 37.

b. Scénario nominal

<i>UTILISATEUR</i>	<i>SYSTÈME</i>
1. L'utilisateur déclenche la procédure pour participer à une soumission	
2. L'utilisateur donne le nom et le mot de passe de la soumission	
3. L'utilisateur donne une clé SSH publique	
	4. Le système vérifie que la combinaison "nom de soumission" et "mot de passe" est correcte
	5. Le système vérifie que la clé SSH publique est valide
	6. Le système crée un subrepository dans le repository de la soumission ¹⁷
	7. Le système limite l'accès au subrepository créé au point précédent avec la clé SSH publique donnée au point 3 ¹⁸
	8. Le système envoie un e-mail à l'utilisateur

c. Scénarios alternatifs

4.A. [LA COMBINAISON "NOM DE SOUMISSION" ET "MOT DE PASSE" EST INCORRECTE]

4.A.1. Branchement au point 2

5.A. [LA CLÉ SSH PUBLIQUE N'EST PAS VALIDE]

5.A.1. Branchement au point 2

¹⁷ Voir *Création d'un subrepository* page 37

¹⁸ Voir *Création d'un accès SSH* page 37

3. Cas d'utilisation « Soumettre une solution »

a. Modèle

Le cas d'utilisation "Soumettre une solution" permet à un étudiant participant à une soumission de lancer l'exécution de tests sur le code qu'il a écrit.

Nom du cas d'utilisation	Soumettre une solution
Objectif	Soumettre une solution
Niveau	Tâche utilisateur
Acteur principal	Etudiant
Parties prenantes et intérêts	
Pré-conditions	L'utilisateur participe à une solution dans le système, il a cloné son subrepository en local grâce à sa clé SSH et il a effectué (au moins) un commit dans ce subrepository
Etat après réussite	Le système a enregistré les résultats des tests sur la solution, et l'instructeur ainsi que l'étudiant peuvent récupérer les résultats des tests
Etat après échec	Le système n'a pas enregistré de résultat aux tests sur la solution et ni l'instructeur ni l'étudiant ne peuvent récupérer les résultats des tests
Trigger	L'utilisateur déclenche la procédure de soumission
Scénario nominal	Voir page 18
Alternatives	Aucune

b. Scénario nominal

<i>UTILISATEUR</i>	<i>SYSTÈME</i>
1. L'utilisateur effectue un push de sa solution	
	2. Le système update le subrepository
	3. Le système lance des tests sur le code du subrepository
	4. Le système exporte les résultats sur l'application web Google associée à la soumission
	5. Le système envoie un e-mail à l'utilisateur

Malgré un scénario nominal d'une longueur très limitée, ce cas d'utilisation est complexe à mettre en œuvre de par les contraintes que nous lui imposons.

Nous souhaitons que Scavenbox ait la capacité d'être utilisé par un nombre très élevé d'utilisateurs. Il nous faut pour cela paralléliser le traitement de l'exécution d'une batterie de tests sur des grappes de serveurs. Les tests pouvant avoir une limite de temps d'exécution, il faut s'assurer que chaque même test ait droit au même temps CPU pour s'exécuter. Il serait gênant que le code d'un étudiant A réussisse un test parce qu'il a été exécuté sur un serveur rapide et que le code d'un étudiant B rate le même test parce qu'il a été exécuté sur un serveur plus lent ou sur un serveur occupé.

Le code d'étudiant que nous testons sur nos serveurs est peut-être mal intentionné. Il faut donc l'empêcher de nuire à l'intégrité de ces derniers.

Ces problématiques seront abordées au chapitre 4, *Outils utilisés*.

4. Outils utilisés

a. Parallélisation de traitement

Les tests exécutés par Scavenbox sont limités dans leur temps d'exécution. Cela est fait d'une part pour ne pas qu'un test appelant du code d'étudiant ne boucle à l'infini et d'autre part pour permettre à un instructeur d'estimer qu'au delà d'un certain temps d'exécution, le code d'un étudiant ne mérite pas ses points. Hors, si plusieurs étudiants soumettent chacun leur solution en même temps, cela lancera la batterie de tests pour chaque solution en même temps également. Nous observons dans ce cas que des étudiants ratent des tests qu'ils auraient normalement dû réussir pour motif de limite de temps dépassée. En effet, étant donné que plus de tests tournaient en même temps, moins de ressources et moins de temps CPU ont été accordés à un seul étudiant.

Pour résoudre ce problème il faut ne pas exécuter toutes les batteries de tests en même temps. Il nous faut un système de queue, où chaque tâche dans la queue est une seule batterie de tests. Ainsi, si la seule chose qui tourne sur notre serveur est l'exécution de tests, et que chaque batterie de tests est exécutée à tour de rôle, les ressources et le temps CPU accordés à une batterie de tests seront toujours les mêmes.

Un des objectifs de Scavenbox est d'être capable de monter en charge. Que l'application soit utilisée par 10 ou par 10 000 étudiants, cela ne doit pas poser de problèmes. En ayant uniquement notre système de queue sur une seule machine, il se pourrait, avec beaucoup d'étudiants, que les étudiants soumettent des solutions plus vite que le serveur ne soit capable de les tester. Il faut donc avoir la possibilité d'utiliser plusieurs serveurs. La queue peut dès lors être partagée entre ces différents serveurs; une tâche de la queue étant envoyée à un serveur dès que ce serveur a terminé d'exécuter une batterie de tests et est donc libre.

HTCondor est un kit de composants logiciel qui permet de faire tout ça. Il n'a été renommé de cette façon qu'en octobre 2012 et c'est toujours son ancien nom, Condor, qui est largement utilisé pour le dénommer. Nous utiliserons donc également son ancien nom dans ce document.

Condor, plutôt que d'utiliser des serveurs à part entière pour la parallélisation du traitement, utilise des nœuds d'exécution. Ces nœuds utilisent une partie prédéfinie des ressources d'un serveur et il est donc possible d'avoir plusieurs nœuds d'exécution par serveur. Condor est déployé en salles des machines à l'UCL et est donc un outil connu. Amazon permet d'utiliser ses machines Amazon EC2¹⁹ en tant que nœuds Condor. Il existe également des machines Amazon EC2 pré-configurées avec Condor.

Rappelons que nous voulons que chaque tâche de la queue ait les mêmes ressources à disposition. Il faut donc que chaque nœud sur chaque serveur soit paramétré avec les mêmes ressources. Pour soumettre une tâche à Condor, il faut lui envoyer un descripteur de tâche. Ce descripteur de tâche ressemble à ce qu'on peut voir sur le *Code 3*.

19 Amazon EC2 : <http://aws.amazon.com/fr/ec2/>

```
vanilla.submit x requirements x
1 universe = vanilla
2 executable = lxc.sh
3 output = lxc.$(cluster).out
4 error = lxc.$(cluster).error
5 log = lxc.log
6 queue
```

Code 3 : descripteur de tâche Condor

```
vanilla.submit x requirements x
1 requirements = Memory >= 32
```

Code 4 : mécanisme de ClassAd de Condor

Ce descripteur assez basique indique quel programme exécuter et dans quels fichiers stocker l'output, les erreurs et les logs de l'exécution. C'est par son intermédiaire que nous lancerons l'exécution des tests. La variable `$(cluster)` indique un numéro de cluster interne à Condor, et qui nous assure que chaque fichier d'output et d'erreurs aura un nom différent.

Le *Code 4* est à ajouter dans un descripteur de tâche au besoin. Il nous montre qu'il est possible de limiter les nœuds sur lesquels l'exécution de la tâche soumise à Condor a le droit d'être exécutée. Supposons par exemple que nous disposions d'un parc informatique avec des nœuds Condor utilisés pour diverses applications. Supposons également que ces nœuds aient des capacités différentes. Il faut pouvoir dire à Condor que notre batterie de tests doit être exécutée par un nœud faisant partie d'un sous-ensemble des nœuds de notre parc informatique. Ce sous-ensemble étant lui composé de nœuds aux capacités identiques. Les expressions définissant les capacités d'un nœud et les exigences d'une tâche sont appelées mécanisme de *ClassAd*.



b. Sandboxing

1. Problématique

Notre dernière problématique concernant l'exécution de tests est que nous exécutons du code qui peut potentiellement faire n'importe quoi et donc nuire à notre système par exemple. Nous voulons bien entendu éviter cela. Il nous faut pour cela un environnement "sandboxé", ou environnement "bac à sable", c'est-à-dire un environnement sécurisé où le code exécuté n'aura pas de répercussions sur notre système et où l'utilisation des périphériques est restreinte.

2. Contrôle d'accès obligatoire avec SELinux et AppArmor

Une première manière de faire est de faire appel au contrôle d'accès obligatoire. Il s'agit d'une méthode de gestion des droits des utilisateurs dans laquelle les décisions de protection ne sont pas prises par le propriétaire des objets concernés. Les décisions dépendent des attributs de sécurité donnés au sujet qui tente d'accéder à un objet ainsi qu'aux attributs de sécurité de l'objet accédé. Un sujet est un processus ou un thread tandis qu'un objet est quelque chose comme un fichier, un dossier, des ports TCP ou UDP, un segment de mémoire partagée, un périphérique d'entrée/sortie, etc.

Pour qu'un sujet puisse effectuer une opération sur un objet, il faut donc que la politique d'autorisations le permette. Nous pourrions, dans notre cas, faire en sorte que nos tests soient limités à un certain dossier et n'aient pas accès à nos périphériques. Bien que cette solution soit envisageable, elle paraissait inutilement complexe à mettre en œuvre et à configurer à cause des nombreux attributs de sécurité à définir.

3. Virtualisation

a. Différentes techniques

J'ai donc été voir du côté de la virtualisation. Elle permet en effet de faire tourner un système d'exploitation complet en tant qu'un simple logiciel. L'idée est alors de faire tourner nos tests dans le système d'exploitation virtualisé. Les tests n'ont pas conscience qu'ils sont dans un environnement virtualisé et n'ont pas accès au système d'exploitation hôte. Cela limite d'emblée les attaques malintentionnées sans avoir besoin d'une lourde phase de configuration.

Il existe différentes techniques de virtualisation présentant chacune des avantages et des inconvénients.

b. Noyau en espace utilisateur avec UML

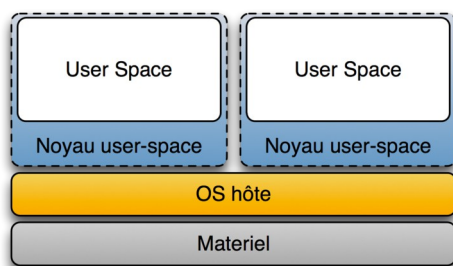


Figure 4 : virtualisation avec noyau en espace utilisateur
Source : Wikipédia

La première technique que j'ai essayé est celle du noyau en espace utilisateur avec User Mode Linux (UML). Cet outil m'a été recommandé car il est utilisé par Pythia.

Cette technique consiste à faire tourner un noyau de système d'exploitation en espace utilisateur, c'est-à-dire en tant que logiciel sur le système d'exploitation hôte. Cela limite complètement les interactions entre le système virtualisé et le système hôte. C'est donc une technique fort sécurisée mais peu performante à cause de l'empilement de 2 noyaux.

De plus, je ne suis pas parvenu à faire fonctionner UML dans une tâche sur Condor. Le démarrage du noyau plantait à chaque fois alors qu'il n'y avait pas de problème pour démarrer un noyau et exécuter des tests dans son espace utilisateur si je ne passais pas par Condor.

Après 2 jours à chercher de la documentation là-dessus, je n'ai trouvé qu'un seul document réunissant les termes UML et Condor.²⁰ A la fin de l'introduction de document, il est question d'utiliser Xen avec Condor plutôt que VMware ou UML car Xen est plus performant. Cela dit je n'ai toujours pas compris pourquoi UML ne fonctionne pas avec Condor.

c. Hyperviseur de type 1 avec Xen et KVM

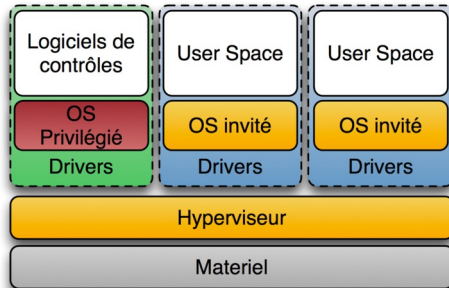


Figure 5 : virtualisation avec hyperviseur de type 1
Source : Wikipédia

J'ai donc entrepris d'utiliser Xen. L'utilisation de ce dernier avec Condor est largement documentée.

Xen est un hyperviseur de type 1, c'est-à-dire une plateforme de virtualisation qui se positionne entre le matériel et les systèmes d'exploitation. Le système d'exploitation hôte, ou privilégié, se situe au même niveau que les systèmes d'exploitation invités, ou virtualisés. Cela permet de grandes performances et une grande sécurité de par l'isolement des systèmes.

Malheureusement, les systèmes d'exploitation qui peuvent tourner sur un hyperviseur de type 1 doivent être adaptés, car ils ont conscience d'être virtualisés. Cela ne nous

convient pas car nous voulons pouvoir déployer Scavenbox sur un serveur avec une distribution Debian classique ou un dérivé (Ubuntu ou autre). Heureusement, les processeurs actuels sont dotés d'instructions de virtualisation matérielle (AMD-V et Intel VT). Mais nous voulons également que Scavenbox puisse être déployé sur un serveur avec un système d'exploitation virtualisé, pour pouvoir être hébergé dans le cloud sur des serveurs Amazon par exemple.²¹ Hors un système d'exploitation virtualisé ne dispose plus des instructions de virtualisation matérielle, ce qui nous ramène au problème de ne pas pouvoir utiliser d'hyperviseur de type 1 avec un système d'exploitation classique.

Il nous est donc impossible d'utiliser Xen mais aussi KVM, un autre hyperviseur de type 1 largement documenté pour son utilisation avec Condor.

d. Hyperviseur de type 2 avec VMware et Vagrant

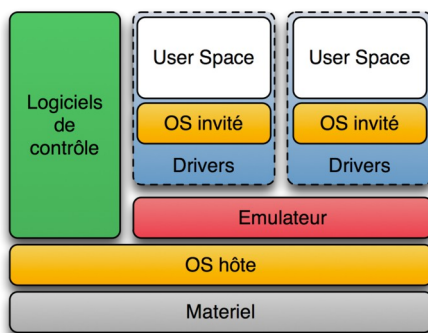


Figure 6 : virtualisation avec hyperviseur de type 2
Source : Wikipédia

Le dernier logiciel de virtualisation bien documenté pour son utilisation avec Condor est VMware, un hyperviseur de type 2. Un hyperviseur de type 2 est un logiciel qui s'exécute à l'intérieur du système d'exploitation hôte. Le grand avantage de ce type d'hyperviseur est qu'il peut virtualiser des systèmes d'exploitation sans que ceux-ci en aient conscience et donc il peut virtualiser n'importe quel système d'exploitation classique.

Le soucis de VMware est qu'il s'agit d'un logiciel propriétaire alors qu'un des objectifs de Scavenbox est de n'utiliser que des logiciels libres. Je me suis donc tourné vers Vagrant, un logiciel permettant d'utiliser VirtualBox en lignes de commandes. J'ai vite abandonné à cause de la lourdeur et de la lenteur du processus. Il nous faut en effet cloner et démarrer

une machine virtuelle complète à chaque fois qu'un étudiant fait un push sur un repository. L'hyperviseur de type 2 n'est pas adapté à cet usage, il convient mieux lorsqu'on a besoin d'une machine virtuelle permanente, que l'on ne doit pas arrêter.

²⁰ Lien vers le document : <http://pages.cs.wisc.edu/~pradheep/SandboxingWorlds05.pdf>

²¹ Amazon EC2 : <http://aws.amazon.com/fr/ec2/>

e. Isolateur avec OpenVZ et LXC

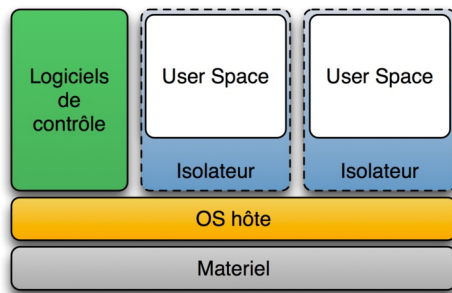


Figure 7 : virtualisation avec isolateur

Source : Wikipédia

C'est avec un soupçon de désespoir que je m'attaque à la dernière technique de virtualisation : l'isolateur. Un isolateur permet d'isoler l'exécution d'un logiciel dans des contextes d'exécution. Cette virtualisation ne peut se faire que sur Linux et l'environnement virtualisé n'est pas complètement isolé, le système virtualisé partageant par exemple son noyau avec le système hôte. Les formes d'isolation sont variées et leur niveau de sécurité l'est donc aussi.

Deux isolateurs semblent sortir du lot : OpenVZ et LXC.

Mon choix s'est porté sur LXC pour sa simplicité d'utilisation.

LXC est le sigle de Linux Containers. Un isolateur dans LXC s'appelle un conteneur. LXC permet de créer, cloner, démarrer, arrêter, supprimer ou exécuter un programme dans un conteneur au simple moyen de commandes spécifiques à chacune de ces actions.

LXC est basé sur la commande *chroot* qui permet de changer le répertoire racine d'un processus. Cela permet d'avoir le système de fichiers complet d'un conteneur présent dans un dossier sur la machine hôte tout en empêchant au conteneur de voir les fichiers de la machine hôte en dehors de ce dossier. Le noyau n'est pas virtualisé mais LXC utilise les *cgroups*, ou control groups, une fonctionnalité du noyau Linux qui permet d'isoler l'utilisation des ressources en les séparants par espaces de nommage. Cela permet de virtualiser l'environnement d'exécution, c'est-à-dire le processeur, la mémoire et le réseau.

Toutes les commandes de LXC doivent être exécutées en tant que root, l'administrateur principal du système. Nous utilisons pour cela l'utilitaire *sudo*, qui permet d'exécuter un programme avec les droits d'un autre utilisateur et le plus souvent l'utilisateur root. Sudo demande un mot de passe lorsque l'on souhaite exécuter un programme en tant qu'un autre utilisateur. Hors nous voulons que ces commandes soient exécutées à partir d'un script bash, c'est-à-dire en l'absence d'une personne physique et en l'absence d'un TTY²². Sudo permet d'autoriser l'exécution de certains programmes par certains utilisateurs en tant que certains autres utilisateurs sans mot de passe. Il faut pour cela éditer le fichier */etc/sudoers* à l'aide de la commande *sudo visudo*, qui édite ledit fichier de manière sécurisée en empêchant que plusieurs personnes ne l'éditionnent en même temps. Le *Code 5* nous montre une partie de ce fichier.

²² TTY, voir glossaire page 47

```

GNU nano 2.2.6                               File: /etc/sudoers.tmp

# Cmnd alias specification
Cmnd_Alias LXC_CLONE = /usr/bin/lxc-clone -o template -n condor_*
Cmnd_Alias LXC_EXECUTE = /usr/bin/lxc-execute -n condor_* -- sh -c cd /job; ./ant_runner.sh *
Cmnd_Alias LXC_DESTROY = /usr/bin/lxc-destroy -n condor_*
Cmnd_Alias CP_TO_LXC = /bin/cp * /var/lib/lxc/condor_*/rootfs/job
Cmnd_Alias RSYNC_TO_LXC = /usr/bin/rsync * /var/lib/lxc/condor_*/rootfs/job
Cmnd_Alias CP_FROM_LXC = /bin/cp /var/lib/lxc/condor_*/rootfs/job/* *
Cmnd_Alias LXC = LXC_CLONE,LXC_EXECUTE,LXC_DESTROY,CP_TO_LXC,RSYNC_TO_LXC,CP_FROM_LXC

# User privilege specification
root    ALL=(ALL:ALL) ALL
submit  ALL=(root) NOPASSWD:LXC

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "#include" directives:

#include /etc/sudoers.d

```

Code 5 : édition du fichier /etc/sudoers avec visudo

Nous voyons que nous pouvons définir une série de *cmnd alias*, qui sont des commandes respectant le globbing. Le globbing permet le pattern matching, ou filtrage par motif. Ici nous utilisons des *** qui font passer le filtrage à n'importe quel nombre de caractères inconnus. Le globbing permet aussi d'utiliser des *?* pour faire passer le filtrage lorsqu'il y a un et un seul caractère inconnu à son emplacement. Il offre également d'autres fonctionnalités plus avancées mais dont nous n'avons pas besoin.

Observons la ligne de code suivante :

```
submit ALL=(root) NOPASSWD:LXC
```

submit indique qu'il s'agit d'une permission accordée à l'utilisateur submit. *ALL* indique que cet utilisateur peut obtenir cette autorisation depuis n'importe quelle adresse IP. *root* indique qu'il ne peut exécuter la commande autorisée qu'en tant que l'utilisateur root. *NOPASSWD* indique que la commande pourra être exécutée sans mot de passe. *LXC* indique quelle est la commande que l'on autorise. Il s'agit ici d'une *cmnd alias* regroupant une série d'autres *cmnd alias*.

Nous pouvons maintenant utiliser les commandes dont nous avons besoin. Nous travaillons avec un conteneur template contenant un système de fichiers Debian avec certaines paquets préinstallés. Chaque fois que nous exécuterons des tests nous clonerons ce conteneur template. Il faudra ensuite rajouter les sources contenant les tests et le code de l'étudiant pour pouvoir exécuter des tests. Pour cela le repository parent de la soumission est placé dans un répertoire */job* dans le conteneur. Est copié également le subrepository de l'étudiant qui a soumis une solution. Une fois les tests exécutés, les résultats sont récupérés dans le conteneur et copiés dans le subrepository de l'étudiant. Le conteneur est enfin supprimé.

5. Frameworks de tests

a. Description

Scavenbox doit permettre de tester du code d'étudiant écrit en n'importe quel langage de programmation. Il faut néanmoins développer au préalable un framework de tests spécifique afin de pouvoir tester du code écrit dans le langage de programmation désiré. Ce framework de tests peut se positionner en surcouche d'un framework XUnit pour plus de facilité mais cela n'est pas une obligation.

Chaque framework de tests se doit de déterminer une durée d'exécution maximale par défaut pour chaque fonction de test. En effet, on ne souhaite pas qu'une boucle infinie tourne sur notre machine indéfiniment. Il faut néanmoins permettre à l'instructeur écrivant ses tests de définir la durée maximale d'exécution pour chacun des tests.

Il faut également offrir la possibilité de donner un score différent à chaque test pour permettre une pondération des points.

De plus, un framework de tests doit outputter deux fichiers : *result* et *details*. Le fichier *result* contiendra une ligne unique reprenant le résultat de l'étudiant. Par exemple :

```
Result: 8/10
```

Le fichier *details* reprendra le détail de l'exécution des tests avec le score obtenu pour chaque test et un traceback en cas d'erreur d'exécution.

Un framework de tests doit aussi fournir une fonction pour outputter un fichier *solution*. Le contenu de ce fichier sera envoyé sur l'application web Google comme étant la solution de l'étudiant à un problème d'optimisation. Le contenu du fichier *result* sera également envoyé sur cette application web Google. Les contenus des fichiers *result* et *details* seront mis ensemble pour former un nouveau fichier ajouté dans les repositories après l'exécution des tests. Le contenu de ce nouveau fichier sera aussi envoyé par e-mail à l'étudiant. Plus d'informations à ce sujet sont disponibles au chapitre *Présentation des résultats* page 38.

Enfin, un framework de tests doit contenir un fichier *test_runner.sh* qui lancera l'exécution des tests de manière appropriée au langage que l'on veut tester. Il doit veiller à ce que les dossiers *src/*, *lib/* et *res/* soient connus lors de l'exécution des tests.

b. Frameworks développés

1. Java

Pour tester du code Java, j'utilise le framework XUnit JUnit²³. Celui-ci permet déjà d'écrire des tests et de limiter le temps d'exécution des tests. Mais le temps d'exécution n'est pas limité par défaut. J'ai donc implémenté une classe *TestBase* avec une règle qui limite par défaut le temps d'exécution de chaque test à une minute. L'instructeur voulant écrire une classe de test devra hériter de celle-ci. En effet seuls les tests des sous-classes de *TestBase* seront exécutés. Cette classe dispose d'une méthode *writeSolution(String solution)* qui pourra être utilisée par l'instructeur dans ses tests pour outputter le fichier *solution*. Cette méthode ne doit être utilisée qu'une seule fois.

²³ Le site de JUnit est disponible à l'adresse suivante : <http://junit.org/>

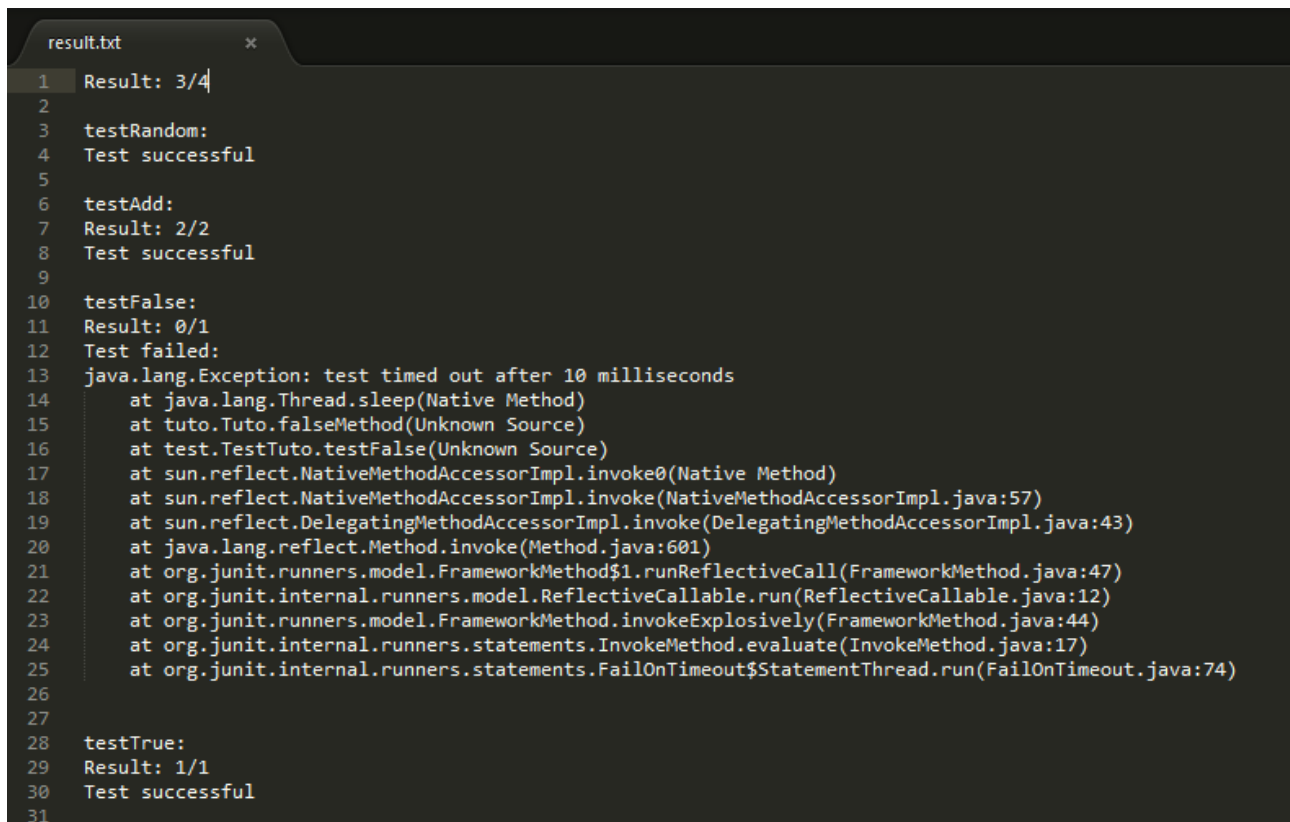
FRAMEWORKS DE TESTS

JUnit ne permet par contre pas de donner un score différent pour chaque test. J'ai donc créé une annotation `@Score` à placer au dessus de chaque méthode de test qui doit valoir des points. Ensuite, plutôt que d'utiliser le lanceur de tests JUnit par défaut, j'utilise ma propre classe pour exécuter les tests un par un. Je peux de cette manière aller chercher l'annotation `@Score` par réflexivité et donc donner un score à chaque test.

JUnit offre des annotations `@Before` et `@After` qui permettent d'exécuter un même code respectivement avant et après chaque tests d'une même classe. Les annotations `@BeforeClass` et `@AfterClass` permettent en théorie d'exécuter du code une seule fois avant et après l'exécution de tous les tests. Elles ont ici le même effet que les annotations `@Before` et `@After` étant donné que chaque test est exécuté un par un.

Le fichier `test_runner.sh` appelle une tâche Ant²⁴. Cette tâche Ant permet de compiler les sources en utilisant bien les dossiers `src/`, `lib/` et `res/`. Une fois les sources compilées, le lanceur de tests qui récupère le score par réflexivité est exécuté. Ce lanceur de tests s'occupe également d'outputter les fichiers `result` et `details`.

Voici un exemple de rapport de tests que l'étudiant pourra recevoir par e-mail :



```
1 Result: 3/4
2
3 testRandom:
4 Test successful
5
6 testAdd:
7 Result: 2/2
8 Test successful
9
10 testFalse:
11 Result: 0/1
12 Test failed:
13 java.lang.Exception: test timed out after 10 milliseconds
14   at java.lang.Thread.sleep(Native Method)
15   at tuto.Tuto.falseMethod(Unknown Source)
16   at test.TestTuto.testFalse(Unknown Source)
17   at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
18   at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:57)
19   at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
20   at java.lang.reflect.Method.invoke(Method.java:601)
21   at org.junit.runners.model.FrameworkMethod$1.runReflectiveCall(FrameworkMethod.java:47)
22   at org.junit.internal.runners.model.ReflectiveCallable.run(ReflectiveCallable.java:12)
23   at org.junit.runners.model.FrameworkMethod.invokeExplosively(FrameworkMethod.java:44)
24   at org.junit.internal.runners.statements.InvokeMethod.evaluate(InvokeMethod.java:17)
25   at org.junit.internal.runners.statements.FailOnTimeout$StatementThread.run(FailOnTimeout.java:74)
26
27
28 testTrue:
29 Result: 1/1
30 Test successful
31
```

Code 6 : exemple de rapport de tests du framework de tests Java

²⁴ Ant, voir glossaire page 46

2. Bash

J'utilise ici le framework de tests XUnit shUnit²⁵. Ce framework ne permet pas de limiter l'exécution des tests dans le temps ni de leur attribuer un score. J'ai défini que ce serait des commentaires avant les tests qui détermineraient ces 2 valeurs de la manière suivante :

```
2
3 # @Timeout 300
4 # @Score 5
5 testEquality() {
6     assertEquals 1 1
7 }
8
```

Code 7 : commentaires pour donner un temps d'exécution et un score à un test pour le framework de tests Bash

Le temps d'exécution maximal de la fonction est donné en millisecondes et est d'une minute s'il est absent. Comme pour le framework de tests Java, j'exécute ici les tests un par un. shUnit propose des fonctions similaires aux annotations JUnit pour exécuter du code respectivement avant et après chaque tests : *setUp()* et *tearDown()*. Les fonctions *oneTimeSetUp()* et *oneTimeTearDown()* qui exécutent du code avant un ensemble de tests ne doivent pas être utilisées. La fonction *suite()* qui permet de définir l'ensemble des fonctions à tester ne doit pas être utilisée non plus. Ce sont toutes les fonctions commençant par *test* dans des fichiers *.sh* commençant eux aussi par *test* qui seront exécutées.

La possibilité est offerte à l'instructeur d'utiliser les fichiers *setUp.sh* et *tearDown.sh* pour exécuter du code avant et après l'ensemble des fichiers de tests. Cela est utile pour installer des dépendances avec *apt-get*²⁶ par exemple.

Voici un exemple de rapport de tests que l'étudiant pourra recevoir par e-mail :

```
1 Result: 5/7
2
3 testEquality
4 Result: 5/5
5 Test successful
6
7
8 testTimeout
9 Result: 0/2
10 Test failed:
11 Timed out after 500 milliseconds
12
13 testSolution
14 Test successful
15
```

Code 8 : exemple de rapport de tests du framework de tests Bash

²⁵ La documentation de shUnit est disponible sur <http://shunit2.googlecode.com/svn/tags/source/2.1.0/doc/shunit2.html>

²⁶ *apt-get*, voir glossaire page 46

6. Gestion des soumissions



a. Serveur Apache2

1. Introduction

Certains de nos cas d'utilisation peuvent se réaliser en lignes de commandes. Mais les cas d'utilisation "Créer une soumission" et "Participer à une soumission" nécessitent que l'utilisateur encode beaucoup de données. Ils sont donc adaptés à l'utilisation d'une interface graphique. Il fallait choisir entre le développement d'une application bureau ou bien d'une application web pour leur réalisation.

Mon choix s'est porté sur une application web, ne nécessitant pas que l'utilisateur télécharge un programme. Il fallait donc ensuite choisir un serveur web. C'est sans hésiter que j'ai choisi d'installer un serveur HTTP Apache dans sa version 2, plus souvent appelé Apache2. C'est un serveur robuste et éprouvé né en 1995.

2. Django

a. Introduction

Nous avons d'abord pensé à réaliser cette application web avec le framework web Django développé en Python. Django fonctionne avec des templates HTML qui sont appelés dans les vues pour être remplis avec les données d'une base de données par exemple.

Ce framework offre un certain nombre de facilités. Le mapping url est décrit dans un fichier unique. Il est possible d'envoyer des mails avec une facilité déconcertante. Le *Code 9* nous montre comment nous pouvons écrire un formulaire HTML pour se connecter à notre compte :

```
urls.py      *  models.py      *  login.py      *
1  # -*- coding: utf-8 -*-
2  from django import forms
3
4  from vdc.settings import DEFAULT_ERRORS
5
6  class LoginForm(forms.Form):
7      login = forms.CharField(label="Identifiant", max_length=100, error_messages=DEFAULT_ERRORS)
8      password = forms.CharField(label="Mot de passe", max_length=30, widget=forms.PasswordInput,
9                                  error_messages=DEFAULT_ERRORS)
```

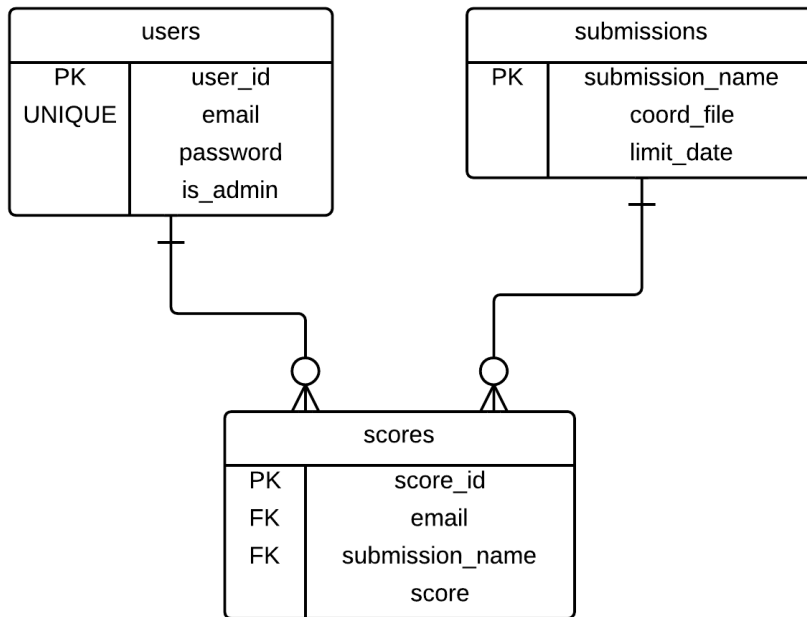
Code 9 : exemple de formulaire avec Django

Nous voyons qu'il suffit d'étendre la classe *Form* de Django et d'y définir les champs que l'on souhaite. Ensuite, pour intégrer ce formulaire dans notre vue, il faut créer un nouvel objet de notre classe *LoginForm* et l'affecter à une variable d'un template prévue à cet effet. Cela créera automatiquement le code HTML du formulaire.

GESTION DES SOUMISSIONS

Pour me faire la main sur ce framework, j'ai fait abstraction du cas d'utilisation "Créer une soumission" et j'ai considéré qu'il n'existait qu'une seule soumission traitant du problème du voyageur de commerce, ou travelling salesman problem que nous abrègerons en TSP pour le reste de ce document. Le TSP est un problème d'optimisation qui consiste en la minimisation de la distance totale parcourue pour relier un ensemble de coordonnées. Il s'agit d'un problème pour lequel on ne connaît pas d'algorithme permettant de trouver une solution exacte en un temps polynomial, c'est-à-dire avec une complexité d'un polynôme ayant ici pour indéterminée le nombre de coordonnées à relier.

b. Schéma entité-relation



Django fonctionne avec un système de classes pour décrire les tables de la base de données. Pour nous faciliter la tâche il existe une classe *AbstractBaseUser* que nous pouvons étendre et qui gèrera pour nous l'encryption du mot de passe et la vérification de la validité de l'e-mail par exemple. Nous l'étendons ici avec une classe *User* pour produire une table *users*. Cette table dispose d'un champ booléen *is_admin* qui nous indique si l'utilisateur est un instructeur ou non. Nous pouvons de cette manière limiter les cas d'utilisation accessibles à l'utilisateur.

Figure 8 : schéma entité-relation de l'application Django

D'une manière générale, il est très simple de définir une table avec Django comme nous le montre le code suivant pour la table *scores* :

```
33
34 class Score(models.Model):
35     email = models.ForeignKey(AUTH_USER_MODEL, to_field='email')
36     submission_name = models.ForeignKey(Submission)
37     score = models.DecimalField(null=True, max_digits=15, decimal_places=3)
38
```

Code 10 : exemple de modèle avec Django

Il suffit d'étendre la classe *Model* et de définir des champs avec leurs attributs. Une fois tous nos modèles définis, on peut demander à Django de générer la base de données correspondante.

django

3. Flask et Google+ API

a. Introduction



Flask

web development,
one drop at a time

Le schéma entité-relation de notre application étant assez simple, nous avons décidé de nous affranchir d'une base de données et de la remplacer par des outils Google. Tout ce dont nous avons besoin est d'identifier des utilisateurs et de stocker des résultats. Pour l'identification des utilisateurs, nous utilisons l'API Google+ et l'utilisateur se connecte à notre application web à l'aide de son compte Google. Il faut donc que tout utilisateur de Scavenbox ait un compte Google. Le stockage des résultats se fait lui dans une Google Spreadsheet stockée sur le Google Drive de l'instructeur ayant créé la soumission concernée.

Notre application web est tout à coup devenue beaucoup plus simple et nous n'avons plus besoin de la puissance du framework web Django. Nous nous limiterons donc à un framework web beaucoup plus léger : Flask, lui aussi développé en Python.

Notre application web est tout à coup devenue beaucoup plus simple et nous n'avons plus besoin de la puissance du framework web Django. Nous nous limiterons donc à un framework web beaucoup plus léger : Flask, lui aussi développé en Python.

Avant de pouvoir utiliser l'API Google+, il faut créer un compte Google spécifique à l'instance de Scavenbox qui tourne sur notre serveur. Une fois connecté avec ce compte, il faut créer un projet sur la Google Developers Console²⁷. Ensuite, sur la console, il faut activer l'API Google+ pour le projet concerné et créer un client ID de type "application web". Il faut rajouter les origines depuis lesquelles l'API pourra être accédée, c'est-à-dire les noms de domaine et l'adresse IP de notre serveur.

b. Google+ Sign-In



OAuth

Pour que l'utilisateur puisse se connecter à notre application avec son compte Google, l'API Google+ fait appel au protocole OAuth 2. Le but de ce protocole est de faire des appels à une API tierce sous une certaine identité. Pour cela, il faut obtenir un access token, ou jeton d'accès, auprès du serveur de l'API tierce. N'importe qui ayant ce jeton en sa possession peut ensuite faire des appels à l'API tierce.

Il existe traditionnellement deux code flows : le client-side flow et le server-side flow. Avec le client-side flow, le client demande un jeton à l'API tierce et le reçoit. Il peut ensuite faire tous les appels qu'il veut avec ce jeton. Avec le server-side flow, le client demande un code d'autorisation à l'API tierce. Ce code est envoyé à notre application qui peut alors l'échanger contre un jeton d'accès grâce à son client ID. Une fois ce jeton reçu, notre application peut faire tous les appels qu'elle veut à l'API tierce sous l'identité du client.

LE ONE-TIME CODE FLOW

Il peut également être intéressant de faire des appels à l'API tierce côté client aussi bien que côté serveur. Pour cela, il serait possible d'utiliser simplement le client-side flow et d'envoyer le jeton d'accès au serveur de notre application une fois ce jeton reçu par le client. Pour des questions de sécurité, Google a préféré développer le one-time code flow, ou flux de code unique qui utilise un code d'autorisation qui n'est valable qu'une seule fois. La *Figure 9* nous montre le processus de connexion. On voit que le client demande et reçoit toujours un jeton d'accès mais qu'il reçoit en plus un code à utilisation unique. Ce code est envoyé au serveur de notre application par le biais d'une requête POST pour ensuite être échangé, grâce au client ID de notre application, contre un jeton d'accès. Le client aussi bien que le serveur sont donc en possession d'un jeton d'accès et peuvent faire des appels au serveur de l'API Google+.

²⁷ La console est disponible à l'adresse suivante : <https://console.developers.google.com/project>

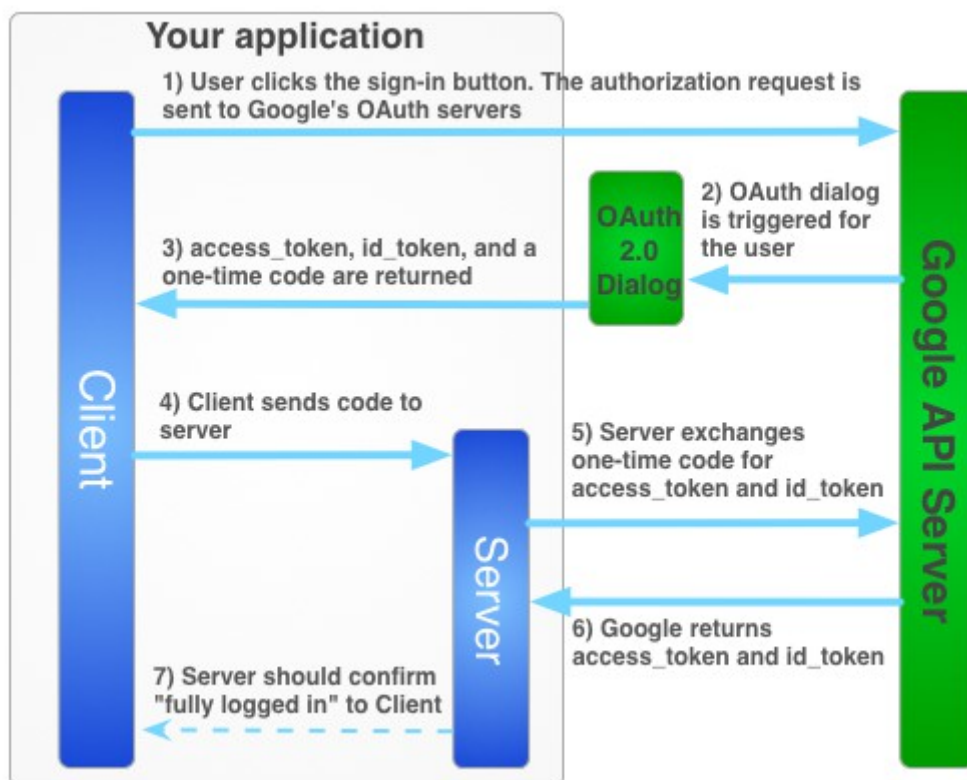


Figure 9 : OAuth : one-time code flow
Source : Google

VULNÉRABILITÉ DU PROTOCOLE

Lisant l'actualité informatique, j'ai eu peur le 2 mai lorsque j'ai lu sur Clubic²⁸ qu'un doctorant de l'université de Singapour avait découvert une faille de sécurité dans le protocole OAuth. Cette faille permettrait à un hacker de rediriger l'utilisateur vers un site frauduleux avant de lui présenter la fenêtre de dialogue OAuth pour se connecter au service souhaité. Le nom de domaine apparaissant dans la barre d'adresse resterait le nom de domaine légitime du site visité, et permettrait au site frauduleux de se faire passer pour celui-ci. Il s'agirait donc d'une technique de phishing permise par ce protocole OAuth, qui donnerait à une personne mal intentionnée le jeton d'accès à l'API tierce que nous cherchons à accéder. Dans notre cas, avec l'API Google+, cela permettrait de récupérer toutes nos informations personnelles, comme notre nom, notre âge, notre sexe, notre liste d'amis, etc.

J'ai été rassuré quelques jours plus tard lorsque j'ai lu sur ZDNet²⁹ que cette faille n'en était en fait pas une. En effet, Elle est apparemment documentée depuis longtemps et il revient à l'organisation implémentant le protocole, Google dans notre cas, d'empêcher les redirections ouvertes. Cela peut se faire en vérifiant l'adresse de redirection avec une liste blanche.

28 L'article est disponible sur <http://pro.clubic.com/it-business/securite-et-donnees/actualite-700058-vulnerabilite-decouverte-oauth-open-id.html>

29 L'article est disponible sur <http://www.zdnet.fr/actualites/covert-redirect-la-faille-pas-grave-d-oauth-et-openid-39800873.htm>

4. jQuery

Notre application web ne doit contenir que 3 pages : la première permettant de créer une soumission, la seconde permettant d'associer un ID d'application web Google une fois celle-ci déployée à une soumission et la 3^{ème} permettant de participer à une soumission. Ces 3 pages étant légères, une autre solution est de faire une seule page contenant 3 onglets gérés en JavaScript. De cette manière, les 3 onglets sont chargés en mémoire au chargement de la page. Le passage d'un onglet à l'autre est alors instantané. C'est cette solution qui a été privilégiée.



Bien que cela soit possible en JavaScript pur, une bonne pratique est d'utiliser la bibliothèque JavaScript jQuery. Cette bibliothèque permet de faciliter la traversée et la manipulation du DOM³⁰, la gestion d'événements, les requêtes Ajax, etc. jQuery offre une API avec une syntaxe beaucoup plus intuitive et moins rébarbative que celle du JavaScript.

5. Mise en page avec Twitter Bootstrap

Twitter Bootstrap est un framework CSS. Il permet de donner du style à nos balises HTML en leur donnant simplement certaines classes de style. C'est ce framework qui donne un style très propre et épuré à Scavenbox :

Figure 10 : application web de Scavenbox

La barre noire en haut, appelée navbar dans le jargon Bootstrap, contient les 3 onglets dont nous avons besoin. Les 2 premiers onglets ne sont accessibles qu'aux instructeurs tandis que le troisième est accessible aux instructeurs et aux étudiants. Chacun de ces onglets contient un formulaire qui appellera un script bash sur le serveur pour effectuer une action spécifique.

³⁰ DOM, voir glossaire page 46

b. Création

1. Introduction

Cette rubrique fait référence au cas d'utilisation "Créer une soumission" dont vous retrouverez le modèle complet à la page 11. L'instructeur voulant mener à bien ce cas d'utilisation doit se connecter à l'application web de Scavenbox et donner un nom de soumission, une tarball gzip (archive) et une clé SSH publique.

2. Création d'un repository

Pour créer une nouvelle soumission, la première chose à faire est de créer un repository propre à cette soumission. Ce repository est créé sur base d'un repository template et de l'archive fournie par l'instructeur. Le repository template contient tout le code nécessaire à l'ajout d'une tâche dans la queue sur Condor, la création d'un conteneur LXC et l'exportation des résultats sur une application web Google. Ce code sera exécuté par les hooks présents dans les subrepositories des étudiants.

L'archive fournie par l'instructeur doit être une tarball gzip pouvant contenir divers fichiers :

- un fichier *language*
- un fichier *Specific.gs*
- un dossier *src/*
- un dossier *lib/*
- un dossier *res/*

Le fichier *language* est obligatoire et déterminera le framework de tests utilisé. Il doit contenir une ligne de texte unique avec le nom du langage du framework de tests que l'on souhaite utiliser. Ces langages sont pour l'instant *java*, *bash* ou *none*.

Le langage *none* est particulier puisqu'il ignore tous les fichiers de la soumission sauf le fichier *Specific.gs*. Le fichier *Specific.gs* sera utilisé pour créer un Google Apps Script et sera facultatif pour les autres langages de soumission. Le contenu de ce fichier est expliqué au point *Contenu spécifique*, page 40. Le langage *none* ne crée aucun repository et ne permet donc pas aux étudiants de s'inscrire à la soumission correspondante.

Le dossier *src/* doit être présent et contenir un dossier *test/* (sauf si le langage de soumission est *none*). Ce dossier *src/test/* doit contenir tous les fichiers de tests écrits par l'instructeur. Le reste du dossier *src/* doit contenir toutes les sources qui seront à disposition de l'étudiant lorsqu'il participera à la soumission. Cela est utile pour lui donner des fichiers contenant la signature des méthodes qu'il devra implémenter ou bien des commentaires expliquant ce qu'il doit faire.

Les dossiers *lib/* et *res/* sont facultatifs. Le dossier *lib/* doit contenir toutes les bibliothèques nécessaires au projet. Le dossier *res/* doit contenir toutes les ressources statiques, comme des images par exemple.

Le code du framework de test sera ajouté dans le repository créé en fonction du langage défini dans l'archive. Il y a 3 types de soumission, sélectionnés automatiquement par le système en fonction des éléments fournis dans l'archive. Les 3 types sont :

- Stand-alone : sélectionné lorsque le langage de soumission est *none*. Ce type de soumission ne passe pas par notre serveur et crée uniquement un Google Apps Script. Ce type de soumission est utile pour les problèmes d'optimisation, comme le TSP, et où l'étudiant peut soumettre une solution directement sur l'application web déployée à partir du Google Apps Script. Cette solution est ensuite testée et un calcul de score est effectué dessus.

- Résultat : sélectionné lorsqu'il n'y a pas de fichier *Specific.gs*. Ce type de soumission passe par notre serveur pour y créer un repository. Seul le résultat des tests est envoyé sur l'application web Google.
- Optimisation : sélectionné lorsque le fichier *Specific.gs* est présent. Ce type de soumission mêle les 2 précédents en exportant le résultat des tests ainsi qu'une solution à tester d'un problème d'optimisation sur l'application web Google.

3. Création d'un accès SSH

Maintenant que nous avons créé un repository, nous voulons que l'instructeur qui est à l'origine de l'action ait accès à ce repository. Nous allons faire cela avec une connexion SSH. Une connexion SSH se fait à un certain compte sur une machine donnée. Il aurait été possible de créer un compte pour chaque instructeur sur la machine hébergeant Scavenbox. Mais cela leur donnerait toujours un accès beaucoup trop complet à notre machine. Nous aimerions donc limiter cet accès à l'exécution des commandes de Mercurial sur le repository nouvellement créé, et avec un seul compte utilisateur pour tous les instructeurs pour plus de facilité. Le protocole SSH se base pour cela sur le fichier `~/.ssh/authorized_keys` qui permet de limiter les commandes qui peuvent être exécutées lorsqu'on se connecte à un compte utilisateur avec une certaine clé SSH. L'utilitaire *hg-gateway* se charge de modifier ce fichier pour nous en y ajoutant la ligne suivante :

```
command="/bin/hg-gateway login email",no-port-forwarding,no-X11-forwarding,no-agent-forwarding clé-ssh
```

Où *email* est remplacé par l'email de l'instructeur et où *clé-ssh* est remplacé par la clé SSH fournie. De plus, si l'instructeur qui crée une soumission en a déjà créé une autre, aucune ligne n'est ajoutée dans le fichier `~/.ssh/authorized_keys` mais *hg-gateway* se charge de garder une liste de repositories accessibles pour chaque adresse e-mail. De cette manière l'utilisateur qui se connectera avec cette clé SSH aura accès à tous les repositories associés à l'e-mail associée à cette clé.

4. Création d'un Google Apps Script

Nous voulons pour chaque soumission une application web Google dédiée pour y présenter la liste des résultats des étudiants. Le Google Drive SDK permet d'exporter des fichiers sur le Google Drive. Nous allons donc utiliser cette fonctionnalité pour exporter les fichiers nécessaires à la création d'un Google Apps Script. Celui-ci sera ensuite déployé en application web. Nous allons aussi créer une Google Spreadsheet qui sera utilisée comme base de données pour le Google Apps Script exporté.

Avant d'utiliser ce SDK, il faut aller sur la Google Developers Console³¹ avec notre compte Google créé pour Scavenbox. Ensuite il faut soit créer un nouveau projet, soit reprendre le projet créé pour notre serveur Apache2 utilisant l'API Google+. Il faut activer la Google Drive API et le Google Drive SDK pour ce projet et créer un client ID de type "application installée".

Le client ID est fourni avec un client secret pour former des credentials. Il faut insérer ces credentials dans le fichier json *client_secrets.json* pour obtenir un JSON Web Token (JWT) avant de pouvoir utiliser le SDK. En effet, comme pour l'API Google+, nous utilisons le protocole OAuth pour pouvoir exécuter des services Google. Mais nous ne voulons pas les exécuter sous l'identité de l'utilisateur, il nous suffit d'exporter l'Apps Script sous l'identité de Scavenbox, de créer une Google Spreadsheet et de changer le propriétaire de ces deux éléments pour l'instructeur qui a créé la soumission.

Le flow OAuth utilisé est comme nous le montre la *Figure 11*.

³¹ La console est disponible à l'adresse suivante : <https://console.developers.google.com/project>

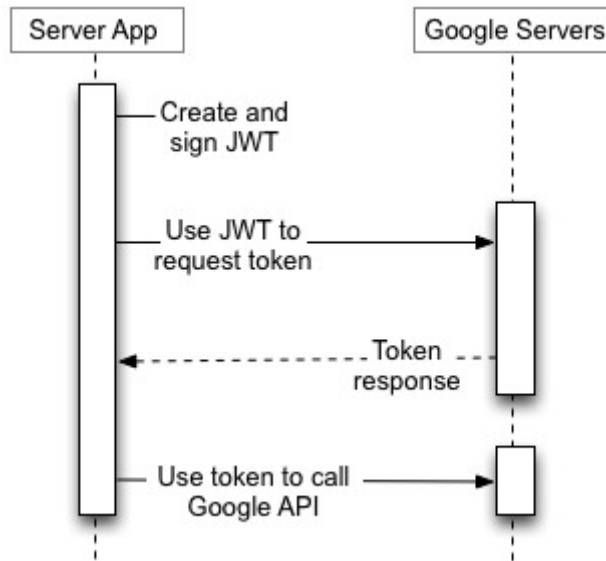


Figure 11 : OAuth : service account flow
Source : Google

Le *Server App* est le serveur de Scavenbox et il ne nécessite aucune interaction avec l'utilisateur.

Google supporte toute une pléthore de langages de programmation pour ses APIs, nous utiliserons ici le langage Python. Le *Code 11* nous montre comment obtenir un objet de services qui nous permet de faire des appels à l'API :

```
80 # If the credentials don't exist or are invalid run through the native client
81 # flow. The Storage object will ensure that if successful the good
82 # credentials will get written back to the file.
83 storage = file.Storage('drive.dat')
84 credentials = storage.get()
85 if credentials is None or credentials.invalid:
86     credentials = tools.run_flow(FLOW, storage, flags)
87
88 # Create an httplib2.Http object to handle our HTTP requests and authorize it
89 # with our good Credentials.
90 http = httplib2.Http()
91 http = credentials.authorize(http)
92
93 # Construct the service object for the interacting with the Drive API.
94 service = discovery.build('drive', 'v2', http=http)
```

Code 11 : obtention des droits à utiliser l'API Google Drive

Nous voyons que le fichier *drive.dat* est utilisé comme jeton JWT. Ce fichier est généré au préalable à partir du fichier *client_secrets.json*. Une fois la validité du jeton vérifiée, nous créons un objet HTTP sur lequel nous autorisons les requêtes aux serveurs de Google grâce à nos credentials. La dernière chose à faire est de créer un objet *service* qui utilisera l'API Google Drive sur notre objet HTTP. Nous pouvons désormais utiliser l'API en appelant des fonctions présentes dans cet objet *service*.

5. Déploiement en application web

Une fois le Google Apps Script et la Google Spreadsheet ayant l'instructeur comme propriétaire, celui-ci peut voir ces documents dans son Drive. La méthode d'identification des fichiers sur le Google Drive se faisant par un identificateur et non par structure de fichiers, l'instructeur peut ranger ces 2 documents dans les dossiers qu'il souhaite.

L'étape suivante est de déployer l'Apps Script en application web. Ce déploiement peut se faire en tant que propriétaire du script ou bien en tant qu'utilisateur accédant à l'application web. Si l'on déploie l'application en tant que l'utilisateur y accédant, c'est-à-dire un étudiant, ce dernier doit alors donner une série d'autorisations au script. Nous préférons éviter ce scénario qui repoussera une partie des étudiants les plus méfiants. Le déploiement du script se fait donc en tant que propriétaire du script, c'est-à-dire un instructeur depuis qu'on a défini celui-ci comme nouveau propriétaire. L'instructeur est bien indiqué comme propriétaire du fichier si on va voir dans les propriétés du fichier. Mais lorsque l'instructeur essaie de déployer le script en application web, le propriétaire est toujours indiqué comme étant Scavenbox. Cela ne nous convient pas puisque Scavenbox n'a pas donné d'autorisation pour l'exécution de ce script et que l'instructeur ne peut pas se connecter au compte Google de Scavenbox pour donner les autorisations au script puisqu'il ne connaît pas le mot de passe de ce compte. Nous ne pouvons pas donner ces autorisations automatiquement par le biais de l'API Google Drive non plus puisque Google requiert que cette opération soit faite manuellement.

Il s'agit là d'un bug de Google qui oublie de changer le propriétaire à un endroit. Je me suis trouvé là dans une impasse ne sachant pas bien comment résoudre le problème. Une solution aurait été de changer de flow OAuth utilisé en nécessitant l'identification de l'utilisateur, identification déjà faite par l'API Google+. Il aurait ainsi été possible de directement exporter les documents sur le Drive de l'instructeur. Cette solution, bien que paraissant la meilleure, aurait été relativement longue à développer. Devant présenter Scavenbox au pôle INGI dans les jours à venir, j'ai décidé de m'orienter vers une solution moins longue à développer. J'ai remarqué que si l'instructeur copiait le Google Apps, il devenait le propriétaire de la copie même lors du déploiement, ce qui règle notre problème. J'ai donc développé un mini Google Apps Script déployé lui aussi en application web et exécuté en tant qu'utilisateur y accédant. L'utilisateur y accédant est un instructeur et doit donner les autorisations au script. Ce script copie le script et supprime l'ancien. Il en profite pour placer la copie du script et la Google Spreadsheet dans un même dossier pour plus de commodité.

Le déploiement du script est ensuite assez simple. Il faut tout d'abord que l'instructeur donne les autorisations nécessaires à l'exécution du script. Il doit ensuite enregistrer une version du script et peut alors déployer cette version.

6. Association de l'application web au repository

Une fois le script déployé en application web, il faut associer cette application web au repository de la soumission, de manière à ce que la requête POST pour exporter les résultats en fin de procédure de tests se fasse sur la bonne URL. Cela se fait en utilisant l'ID de l'application web Google récupérable lors du déploiement. La figure ci-contre nous montre une URL d'application déployée. Le texte surligné en noir jusqu'à la prochaine / représente cet ID.

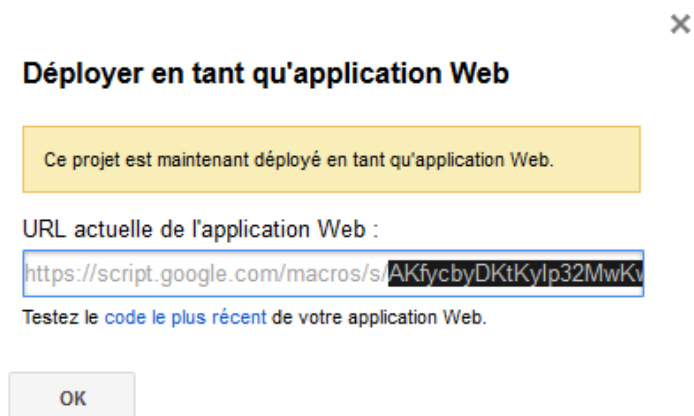


Figure 12 : déploiement d'un Google Apps Script en application web
Source : Google

Une fois l'ID récupéré, il faut retourner sur l'application web de Scavenbox et associer cet ID avec la soumission créée. Il faut également donner un mot passe qui a été envoyé par e-mail à l'instructeur. Ce mot de passe est utilisé pour empêcher un instructeur de donner un identifiant à une soumission d'un autre instructeur, ne serait-ce que par erreur.

c. Modification

Cette rubrique fait référence au cas d'utilisation "Modifier une soumission". L'instructeur ayant créé une soumission a désormais accès à un repository en SSH. Il peut récupérer ce repository à l'aide de la commande suivante :

```
hg clone ssh://submit@adresse-ip/nom-de-soumission
```

Il devra bien entendu ne pas oublier de remplacer *adresse-ip* par l'adresse IP ou le nom de domaine du serveur sur lequel tourne Scavenbox et *nom-de-soumission* par le nom de la soumission qu'il a créé. Il pourra ensuite modifier ses tests et faire un push pour les mettre à jour sur le serveur. Tous les étudiants qui testeront leur solution par la suite auront droit aux nouveaux tests. Si l'instructeur souhaite que le classement présenté sur l'application web Google soit toujours cohérent, il devra aller sur son Google Drive et vider manuellement la Google Spreadsheet de la soumission. Attention, il s'agit bien de la vider et non pas de la supprimer.

d. Participation

1. Introduction

Cette rubrique fait référence au cas d'utilisation "Participer à une soumission" dont vous retrouverez le modèle complet à la page 15. L'étudiant doit fournir le nom de la soumission à laquelle il souhaite participer, le mot de passe de la soumission, ainsi qu'une clé SSH publique. Le mot de passe de la soumission doit avoir été communiqué par l'instructeur à ses étudiants au préalable.

2. Création d'un subrepository

Un subrepository au nom de l'étudiant est créé dans le repository de la soumission. Dans ce subrepository est copiée toute la structure de fichiers du dossier *src/* présent dans le repository parent à l'exception du sous-dossier *src/test/*. De plus, un hook sur le push est ajouté dans ce subrepository pour exécuter les tests du repository parent sur le code de ce subrepository.

Je n'avais au départ pas connaissance de l'existence des subrepositories et j'avais donc implémenté Scavenbox sans ceux-ci. Les repositories des soumissions étaient tous dans un même dossier tandis que les repositories des participations aux soumissions étaient dans un dossier propre à la soumission. Bien que cela fonctionne aussi, la centralisation permise par l'utilisation des subrepositories nous est d'une grande aide.

3. Création d'un accès SSH

Comme pour le repository créé pour l'instructeur, il est intéressant de créer un accès SSH au subrepository de l'étudiant. Nous utilisons toujours *hg-gateway* pour ce faire. Nous donnons bien entendu accès à ce subrepository à l'étudiant mais aussi à l'instructeur responsable de la soumission. En effet, *hg-gateway* ne donne pas accès aux subrepositories même s'il on a accès au repository parent.

L'étudiant peut récupérer le contenu de son subrepository nouvellement créé avec la commande suivante :

```
hg clone ssh://submit@adresse-ip/nom-de-soumission/e-mail
```

En n'oubliant pas de remplacer les champs en gras par des valeurs adéquates.

7. Présentation des résultats

a. E-mail

Une fois que la solution d'un étudiant a été entièrement testée, il faut présenter ses résultats à l'étudiant. Scavenbox présente les résultats sous différentes formes. La première et la plus évidente d'entre elles est l'envoi d'un e-mail contenant un rapport détaillé de l'exécution des tests à l'étudiant concerné.

Les e-mails sont envoyés à travers un serveur SMTP. Scavenbox utilise pour l'instant le serveur SMTP de l'UCL mais il ne faut pas oublier d'en renseigner un autre si Scavenbox est installé dans une autre entreprise. Les e-mails sont envoyés pour d'autres cas d'utilisation que celui-ci. Notamment lors de la création d'une soumission par un professeur où le mot de passe de la soumission est une partie de contenu de l'e-mail envoyé. Il serait dérangeant que quelqu'un intercepte cet e-mail et donc le mot de passe. C'est pourquoi il est préférable d'utiliser la Transport Layer Security (TLS).



TLS est un protocole de sécurisation des échanges sur internet. Il identifie les machines selon un mode de fonctionnement client-serveur suite à un échange de certificats. Les données sont ensuite transmises de manière chiffrée. Ce protocole succède au protocole SSL et est très largement utilisé, dans les transactions bancaires notamment.

En avril dernier, une vulnérabilité importante nommée Heartbleed³² a été découverte. Celle-ci permet à un hacker de récupérer des données présentes sur le serveur et qui ne lui étaient pas destinées. La faille est maintenant corrigée mais elle a secoué le monde de l'informatique et nous a montré une fois de plus qu'il ne fallait pas avoir une confiance aveugle en nos protocoles de sécurité.

b. Application web Google

1. Introduction

Cette rubrique fait référence au cas d'utilisation "Comparer mes résultats avec ceux des autres étudiants". Rappelons que nous déployons des Google Apps Script en applications webs. Le contenu de ces scripts est décrit dans cette rubrique. Ils sont composés d'une partie générique, identique pour tous les scripts, d'une partie spécifique et d'une partie générée par le serveur de Scavenbox avant leur exportation sur le Google Drive.

2. Contenu générique

a. Google Apps Script

Google Apps Script (GAS) est un langage de script développé par Google et basé sur le JavaScript. Il est hébergé par Google et permet d'interagir avec les services Google comme le Drive par exemple. Il offre des facilités de création d'interfaces graphiques en utilisant le Google Web Toolkit (GWT). Contrairement au JavaScript, le GAS est exécuté côté serveur, c'est-à-dire chez Google, et non côté client.

Chacun des GAS que nous devons exporter sur le Drive contient une base de code commune. Ce code sert à gérer les requêtes GET et POST faites à l'application web déployée à partir du GAS. Les requêtes GET sont dans notre cas utilisées par l'utilisateur accédant à l'application web pour afficher le contenu de la Google Spreadsheet. Cette Spreadsheet contient deux onglets : un onglet contenant l'historique de tous les

32 Un article est disponible sur <http://pro.clubic.com/it-business/securite-et-donnees/actualite-695188-heartbleed.html>

PRÉSENTATION DES RÉSULTATS

tests exécutés et un autre onglet contenant un classement reprenant le meilleur résultat de chaque adresse e-mail (c'est-à-dire de chaque étudiant) dans l'historique. Les requêtes POST sont utilisées par Scavenbox lorsque l'exécution de tests est terminée pour rajouter les résultats obtenus dans la Google Spreadsheet de la soumission.

On peut en GAS gérer les requêtes GET avec une fonction *doGet* et faire appel à un template HTML de la manière suivante :

```
192
193 function doGet(e) {
194   var t = HtmlService.createTemplateFromFile('index');
195   return t.evaluate().setTitle(SpreadsheetApp.openById(ssId).getName());
196 }
197
```

Code 12 : fonction *doGet* d'un Google Apps Script

GAS possède un service HTML qui permet de desservir des pages web qui peuvent interagir avec les fonctions du GAS qui seront exécutées côté serveur. Cela peut poser problème d'un point de vue sécurité puisque cela permet d'appeler toutes les fonctions serveur du côté client. Pour ne pas qu'une fonction soit visible côté client, elle doit se terminer par un `_`, de la manière suivante :

```
function nomDeFonction_() { }
```

Les fonctions serveur sont appelées en HTML de la manière suivante :

```
<?!= nomDeFonction_(); ?>
```

Et en JavaScript de cette façon :

```
google.script.run.withSuccessHandler(fonctionEnCasDeSuccès)
  .withFailureHandler(fonctionEnCasDErreur).nomDeFonction_();
```

Ces fonctions sont appelées en HTML pour retourner une chaîne de caractères à intégrer dans le code HTML de la page à afficher. Elles sont appelées en JavaScript pour être appelées dynamiquement lors d'un événement sur la page ou bien si la page a besoin de données accessibles uniquement par le biais du GAS. Il s'agit de requêtes Ajax, qui ne nécessitent pas de manipulation de l'utilisateur pour être envoyées. GAS nous permet de définir un comportement en cas de succès et un autre en cas d'échec de la requête Ajax.

Les 3 types de soumission donnent lieu à des comportements différents dans le code. J'évite de mettre des conditionnelles partout pour ne pas rendre le code illisible. J'utilise par contre le *pattern strategy* qui permet d'exécuter du code différent en fonction de la valeur d'une variable. Ici la variable *type* qui contient le type de soumission du GAS.

b. jQuery et Twitter Bootstrap

Tout comme pour l'application web de Scavenbox, nos GAS utilisent le framework JavaScript jQuery et le framework CSS Twitter Bootstrap.

Les GAS déployés en applications web sont d'abord compilés par Caja, un projet Google qui empêche les objets compilés d'interagir directement avec le DOM de la page. Ils auront besoin d'une référence à un objet enveloppant un objet du DOM pour y accéder.

Certaines fonctionnalités HTML, CSS ou JavaScript ne sont pas compilables avec Caja. Par exemple, j'aurais voulu faire une navbar Bootstrap en position fixée en haut de la page, c'est-à-dire qui reste visible à l'écran même s'il on descend dans la page. Cela est théoriquement possible avec la propriété CSS *position*

PRÉSENTATION DES RÉSULTATS

à laquelle on donnerait la valeur *fixed*, comme le fait Bootstrap. Mais Caja n'accepte pas cette valeur pour cette propriété. Je n'ai donc pas pu faire en sorte que la navbar reste en haut de l'écran en permanence.

Voici l'apparence d'un GAS déployé en application web pour une soumission de type "optimisation" :

Log				Ranking				
#	First name	Last name	Email	Date	Hour	Result	Score	
1	Lionel	Chalet	lionel.chalet@gmail.com	14/05/2014	12:35:10	3/4	1	Download
2	Alfred	Dupont	alfred.dupont@gmail.com	14/05/2014	12:34:14	3/4	9	Download
3	Lionel	Chalet	lionel.chalet@gmail.com	14/05/2014	12:31:24	3/4	1	Download
4	Lionel	Chalet	lionel.chalet@gmail.com	14/05/2014	12:24:04	3/4	7	Download
5	Alfred	Dupont	alfred.dupont@gmail.com	14/05/2014	12:19:17	3/4	3	Download
6	Lionel	Chalet	lionel.chalet@gmail.com	14/05/2014	12:17:25	3/4	6	Download
7	Lionel	Chalet	lionel.chalet@gmail.com	14/05/2014	12:14:49	3/4	8	Download
8	Lionel	Chalet	lionel.chalet@gmail.com	14/05/2014	12:10:07	3/4	10	Download

Figure 13 : Google Apps Script déployé en application web

Les lignes surlignées en bleu sont les lignes concernant le visionneur de la page. Le bouton *Download* à droite n'existe que si c'est l'instructeur qui a créé la soumission qui visionne la page. Il permet de télécharger la solution de l'étudiant à l'origine du calcul du score. Il s'agit de la solution au problème d'optimisation envoyée dans une requête POST par les serveurs de Scavenbox.

Si le type de soumission retenu dans la variable *type* est "stand-alone", ce ne sont pas les serveurs de Scavenbox mais directement l'utilisateur qui doit fournir une solution. Il y a donc un bouton pour uploader un fichier sur le GAS qui apparaît dans la navbar :

Log	Ranking	Submit a file
-----	---------	-------------------------------

Figure 14 : bouton de soumission sur un GAS pour le type "stand-alone"

3. Contenu spécifique

a. Description

Nous avons vu que le professeur voulant créer une soumission a la possibilité d'exporter un fichier *Specific.gs*. Ce fichier contient le code du GAS spécifique à la soumission. Il doit impérativement contenir 4 fonctions. Voici le détail de ces fonctions :

VALIDATION

```
function validate_uploadedFileContent() { }
```

Cette fonction reçoit la solution de l'étudiant au problème d'optimisation sous forme d'une chaîne de caractères. Elle doit renvoyer un booléen déterminant si la solution est considérée comme valide et si une ligne sera ajoutée à l'onglet d'historique du GAS ou non.

ANALYSE

```
function parse_uploadedFileContent() { }
```

PRÉSENTATION DES RÉSULTATS

Cette fonction reçoit en paramètre la même chaîne de caractères que la fonction de validation. Elle n'est appelée que si la fonction de validation renvoie *vrai*. Son rôle est d'analyser la chaîne de caractères et de renvoyer un objet exploitable à partir de celle-ci. (un tableau par exemple)

CALCUL DU SCORE

```
function score_(parsedObject) { }
```

Cette fonction reçoit en paramètre l'objet renvoyé par la fonction d'analyse. Son rôle est de calculer un score à partir de cet objet. Elle renvoie donc un nombre à virgule flottante.

SENS DU TRI

```
function ascending_sort_() { }
```

L'onglet de classement du GAS garde le meilleur résultat de chaque adresse e-mail de l'onglet d'historique et dresse un classement à partir de là. Il reste à déterminer si le score obtenu par la fonction de calcul du score est considéré comme meilleur s'il est plus petit ou plus grand. Cette fonction renvoie un booléen *vrai* en cas de meilleur score lorsque celui-ci est plus petit et *faux* sinon.

b. Exemples

TSP

Le TSP est, rappelons-le, un problème d'optimisation consistant en la minimisation de la distance totale parcourue pour relier un ensemble de coordonnées. Cet ensemble de coordonnées est unique pour un GAS. S'il on souhaite calculer un score pour plusieurs ensembles de coordonnées il faudra créer autant de GAS (c'est-à-dire de soumission avec le langage *none*) que d'ensembles de coordonnées.

Ces ensembles de coordonnées doivent être exportés manuellement par l'instructeur souhaitant les utiliser. Leur contenu peut ensuite être lu dans le fichier *Specific.gs* de la manière suivant :

```
var coordsFileContent = DocsList.getFileById("ID").getContentAsString();
```

La variable *coordsFileContent* contiendra alors le contenu du fichier exporté sur le Google Drive avec l'identifiant *ID*, soit un ensemble de coordonnées.

C'est de cette façon que j'ai implémenté un exemple de fichier *Specific.gs* pour le TSP. J'ai pris comme convention que le fichier de coordonnées sur le Drive contienne 2 nombres à virgule flottante sur chaque ligne. De cette manière chaque ligne représente une coordonnée avec ses composantes *x* et *y*. La convention pour la solution fournie par l'étudiant est qu'il s'agit d'un fichier contenant un numéro sur chaque ligne. Ce numéro correspond au numéro de ligne de la coordonnée dans le fichier de coordonnées sur le Drive.

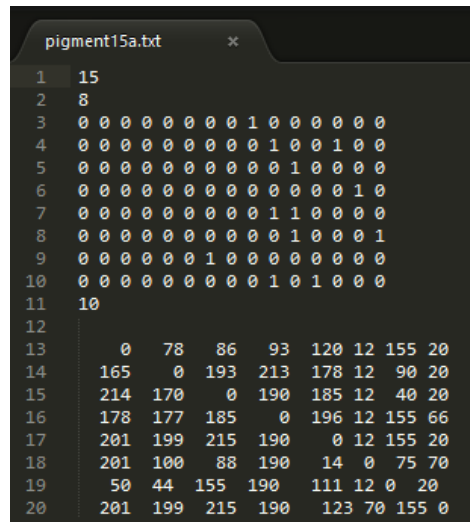
La fonction de validation vérifie que le fichier fourni par l'étudiant soit bien formé de numéros sur chacune de ses lignes. De plus elle vérifie que ces numéros reprennent l'entièreté des numéros de lignes du fichier de coordonnées sur le Drive, sans trou ni doublon.

La fonction d'analyse transforme la liste des numéros de lignes en le tableau de coordonnées correspondant. La fonction de score peut ensuite facilement calculer la distance totale à parcourir pour traverser toutes ces coordonnées. La fonction de sens du tri renvoie *vrai* pour indiquer un tri ascendant car une distance plus petite est considérée comme un meilleur score.

PRÉSENTATION DES RÉSULTATS

INSTANCES PIGMENT SEQUENCING

Ce problème consiste en la minimisation d'un coût total de production de produits sur une chaîne de production. Tout comme pour le TSP, il dépend d'un fichier que l'instructeur aura mis manuellement sur son Google Drive au préalable. Ce fichier ressemble à ceci :



```
pigment15a.txt
1 15
2 8
3 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
4 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0
5 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
6 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
7 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0
8 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1
9 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
10 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0
11 10
12
13 0 78 86 93 120 12 155 20
14 165 0 193 213 178 12 90 20
15 214 170 0 190 185 12 40 20
16 178 177 185 0 196 12 155 66
17 201 199 215 190 0 12 155 20
18 201 100 88 190 14 0 75 70
19 50 44 155 190 111 12 0 20
20 201 199 215 190 123 70 155 0
```

Figure 15 : exemple de fichier contenant des besoins et des coûts de production

Sur la première ligne se trouve le nombre de jours dont on dispose pour produire tous nos produits. Le chiffre sur la deuxième ligne correspond au nombre de produits différents que l'on peut produire. Les prochaines lignes sont une matrice avec des 1 et des 0. Il y a une ligne par produit et une colonne par jour. Cette matrice correspond à nos besoins de production. Un 1 représente le besoin de produire tel produit pour tel jour tandis qu'un 0 représente l'absence de ce besoin. Le nombre sous cette matrice correspond au coût de stockage journalier d'un produit. C'est-à-dire qu'on essaye de produire un produit pas avant d'en avoir besoin sinon il nous coûte de l'argent pour le garder en stock. La dernière matrice correspond aux coûts de modification des machines de la chaîne pour passer de la production d'un produit à la production d'un autre produit.

Le fichier de soumission d'un étudiant reprend une liste composée d'autant de nombres que de jours disponibles. Ces nombres correspondent au numéro du produit que l'on souhaite produire à un jour donné. Un nombre négatif signifiera qu'on ne produira rien ce jour là. La fonction de validation vérifie que les nombres donnés correspondent bien aux besoins de production et que l'on produit le bon nombre de chaque produit. Elle vérifie également qu'un produit ne soit pas produit après la date où l'on en a besoin.

La fonction d'analyse transforme simplement la chaîne de caractères comprenant les nombres correspondant aux jours de production des produits en un tableau de nombres. Le calcul du score est ensuite relativement simple. Un coût plus faible est considéré comme meilleur et la fonction de sens du tri renvoie donc *vrai* pour indiquer un tri ascendant.

4. Contenu généré

Avant d'être exporté sur le Google Drive, un GAS se voit attribuer quelques attributs par Scavenbox. Le GAS doit en effet connaître l'identifiant de la Google Spreadsheet créée, le type de soumission auquel on a affaire, ainsi que l'adresse e-mail du professeur ayant créé la soumission (pour lui présenter le bouton de téléchargement des solutions de ses étudiants).

c. Mercurial

Cette rubrique fait référence aux cas d'utilisations "Récupérer les solutions des étudiants", "Récupérer les résultats des étudiants" et "Récupérer mes résultats". Ces trois cas d'utilisation fonctionnent de la même manière : des résultats sont ajoutés dans les repositories, des commits sont effectués automatiquement et ensuite l'utilisateur peut cloner ou faire un pull du repository.

Une fois les tests terminés, un fichier contenant le détail des résultats est placé dans le subrepository de l'étudiant et est copié dans un dossier contenant les résultats de tous les étudiants dans le repository parent de la soumission. De cette manière, il est accessible pour l'étudiant et de manière centralisée pour l'instructeur. Le dossier reprenant tous les résultats comprend également un fichier `_results` qui reprend les derniers résultats de chaque étudiant. Juste son score, pas le détail de l'exécution des tests. Les lignes de ce fichier sont classées par ordre alphabétique des noms de famille et ensuite des prénoms des étudiants.

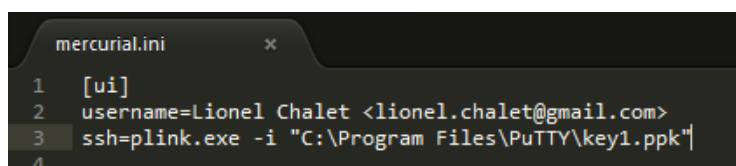
Avant que l'étudiant ou l'instructeur puissent récupérer ces résultats, Scavenbox doit effectuer un commit autant dans le subrepository que dans le repository parent. En effet un commit du repository parent ne commit pas automatiquement ses subrepositories, à moins de préciser l'option `--subrepos`. Une fois cela effectué, l'étudiant ou l'instructeur peuvent faire des pulls et des updates pour récupérer ces fichiers de résultats.

Rappelons qu'avant de pouvoir faire un pull il faut avoir cloné le repository à l'aide de la commande suivante :

```
hg clone ssh://submit@adresse-ip/nom-de-soumission(/e-mail)
```

Pour cloner le repository parent il faut remplacer *adresse-ip* par l'adresse ip ou le nom de domaine de la machine sur laquelle tourne Scavenbox et remplacer *nom-de-soumission* par le nom de la soumission souhaitée. Pour cloner un subrepository il faut y ajouter l'adresse *e-mail* du subrepository que l'on veut. Cette commande fonctionne de base sur les systèmes UNIX (Linux ou Mac) où bien nécessite l'installation du paquet open SSH avec le gestionnaire de paquets de la distribution utilisée. Elle est un petit peu plus délicate à faire fonctionner sur les systèmes Windows, utilisés probablement par la plupart des étudiants.

Il faudra tout d'abord installer PuTTY³³, un client SSH pour Windows. Il faut ensuite installer Plink qui permet d'utiliser PuTTY en lignes de commandes. Il faut enfin aller dans le fichier `mercurial.ini` présent dans le dossier de l'utilisateur Windows. Ce fichier contient les options de Mercurial propres à l'utilisateur :



```
mercurial.ini
1 [ui]
2 username=Lionel Chalet <lionel.chalet@gmail.com>
3 ssh=plink.exe -i "C:\Program Files\PuTTY\key1.ppk"
4
```

Figure 16 : fichier de configuration de Mercurial

Voyons qu'on y a ajouté la ligne

```
ssh=plink.exe -i "chemin vers une clé SSH"
```

dans la section `ui`. On voit que Plink se trouve dans le PATH pour plus de facilité et qu'il faut renseigner le chemin vers la clé SSH qui sera utilisée lors de l'usage de la commande `hg clone ssh`. Cette clé peut être générée avec PuTTYgen.

33 PuTTY & co. sont disponibles à l'adresse <http://www.chiark.greenend.org.uk/~sgtatham/putty/download.html>

Conclusion

EN RÉSUMÉ

L'exécution des tests avec Scavenbox est relativement simple puisqu'il suffit d'effectuer un push sur un repository pour la démarrer.

La parallélisation du traitement est correctement réalisée grâce à Condor et permet à Scavenbox de monter en charge à souhaits.

Il est possible d'implémenter un framework de tests pour n'importe quel langage et il est donc possible de tester n'importe quel langage.

Le code étudiant exécuté sur nos machines est exécuté dans un conteneur LXC et ne risque pas d'endommager notre serveur.

Scavenbox utilise des outils existant et ne réinvente pas la roue. Ces outils sont tous libres.

En résumé nous constatons que tous les objectifs fixés ont été atteints.

Ce stage m'a apporté beaucoup de connaissances. J'ai en effet découvert un très grand nombre de technologies. Ces technologies sont soit très utilisées dans le monde professionnel comme les logiciels de gestion de version décentralisés, Condor, Django ou encore Apache2, soit fort dans l'aire du temps comme les services Google qui font partie du mouvement plus général du Cloud Computing³⁴.

La seule chose qui pourrait être vue de manière négative dans ce stage est que je n'ai pas vu le monde de l'entreprise ni travaillé en équipe.

CHOIX DU NOM DE L'APPLICATION

J'ai voulu faire ressortir les technologies utilisées dans le choix du nom d'application *Scavenbox*.

Scaven vient du mot anglais *scavenger* qui signifie charognard. En effet, notre logiciel de parallélisation de traitement se nomme Condor, qui est un oiseau charognard.

La dernière syllabe *box* provient du terme *sandbox* qui indique que les tests sont exécutés dans un environnement sécurisé.

34 Cloud Computing, voir glossaire page 46

LIMITES ET AMÉLIORATIONS POSSIBLES

Bien que Scavenbox atteigne tous ses objectifs, cela ne signifie pas qu'il n'est pas possible de l'améliorer.

Il n'y a pour l'instant que 2 frameworks de tests développés : un pour Java et un autre pour Bash. Il serait intéressant d'en développer davantage.

Les étudiants ont la possibilité de se partager des repositories entre eux pour travailler à plusieurs sur un même projet. Néanmoins Scavenbox ne retient qu'un seul étudiant par soumission et c'est donc toujours le même membre du groupe qui doit faire des pushes pour lancer l'exécution des tests. Cela n'est pas convaincant et il serait judicieux qu'une participation à une soumission sur Scavenbox puisse être rejointe par d'autres étudiants.

Le serveur Apache2 est actuellement en HTTP. Il serait intéressant de le passer en HTTPS pour ne pas que les mots de passes entrés par les utilisateurs puissent être interceptés facilement. Cela n'est probablement pas très compliqué à configurer.

Actuellement, n'importe qui peut faire une requête POST sur un Google Apps Script déployé en application web. De faux résultats peuvent donc en théorie y être ajoutés. Cela n'est pas très important car cette application web n'est là qu'à titre indicative et ne devrait donc jamais être utilisée par un instructeur pour coter ses étudiants. Il devra utiliser les fonctionnalités de Mercurial comme vu à la page 43 pour récupérer les résultats des étudiants. Mais il serait tout de même mieux de limiter la possibilité de faire des requêtes POST depuis le serveur de Scavenbox uniquement. Cela peut se faire en retenant l'adresse IP du serveur dans le GAS ou bien en utilisant un système de signature chiffrée.

Certains cas d'utilisation supplémentaires gagneraient à être implémentés. Il serait intéressant que l'instructeur puisse supprimer une soumission et lister toutes ses soumissions. L'étudiant, lui, pourrait savoir lister toutes ses participations à des soumissions.

Glossaire

Ce glossaire reprend les termes du présent document qui gagnent à être définis. Ils sont référencés par des notes de bas de page lors de leur première apparition.

AJAX

Une requête Ajax, pour Asynchronous JavaScript and XML, est une requête qui ne nécessite pas de manipulation de l'utilisateur pour être envoyée.

ANT

Ant est un logiciel qui vise à automatiser les opérations répétitives du développement de logiciels telles que la compilation, la génération de documents (Javadoc) ou l'archivage au format JAR.

APT-GET

APT, ou Advanced Packaging Tool, est un gestionnaire de paquets utilisé par la distribution Debian et ses dérivés. *apt-get* est la commande qui permet la gestion de paquets, leur installation entre autres.

CLOUD COMPUTING

Le cloud computing désigne un ensemble de processus qui consiste à utiliser la puissance de calcul et/ou de stockage de serveurs informatiques distants à travers un réseau.

DOM

Le DOM, ou Document Object Model, est un standard du W3C qui décrit une interface indépendante de tout langage de programmation permettant à des programmes informatiques d'accéder ou de mettre à jour le contenu, la structure ou le style de documents XML et HTML.

FRAMEWORK

Un framework est un ensemble cohérent de composants logiciels structurels, qui sert à créer les fondations ainsi que les grandes lignes de tout ou d'une partie d'un logiciel.

GOOGLE APPS SCRIPT

Google Apps Script est un langage de programmation propre à Google. Il permet de réaliser des scripts à l'intérieur de certains services Google.

HOOK

Un hook sur un repository est l'exécution automatique de code lors d'une certaine action effectuée sur ce repository.

REPOSITORY

Un repository est un stockage centralisé et organisé de données. Ce peut être une ou plusieurs bases de données où les fichiers sont localisés en vue de leur distribution sur le réseau, ou bien un endroit directement accessible aux utilisateurs.

SOUMISSION

Il s'agit d'un terme ambigu pouvant avoir 2 significations :

- projet auquel on peut soumettre des solutions
- action de soumettre une solution à une soumission du premier type

SUBREPOSITORY

Un subrepository est un repository créé à l'intérieur d'un autre repository.

TTY

Un TTY (teletypewriter en anglais), est un téléscripteur. Il permet l'encodage et la réception de messages.

Bibliographie

Beaucoup de sites internet ont été utilisés pour la rédaction de ce document. Ceux qui sont indiqués en notes de bas de page dans le document ne sont pas repris ici. La date de dernière consultation pour tous ces sites est le 1^{er} juin 2014.

DJANGO

- <http://www.djangobook.com/en/2.0/index.html>

GOOGLE

- <https://developers.google.com/+web/signin/server-side-flow>
- <https://developers.google.com/accounts/docs/OAuth2>
- <http://gusclass.com/blog/2013/03/22/using-the-hybrid-clientserver-flow-with-google-sign-in/>
- <https://developers.google.com/apps-script/guides/html/>

HTCONDOR

- <http://research.cs.wisc.edu/htcondor/>
- http://research.cs.wisc.edu/htcondor/manual/v7.7.2/2_11Virtual_Machine.html#SECTION00311320000000000000
- http://research.cs.wisc.edu/htcondor/manual/v8.0.3/3_14Virtual_Machines.html#SECTION00414100000000000000

MERCURIAL

- <http://mercurial.selenic.com/wiki/MercurialApi>
- <http://hgbook.red-bean.com/read/handling-repository-events-with-hooks.html>
- <http://www.codza.com/mercurial-with-ssh-setup-on-windows>

SUDOERS

- <http://ubuntuforums.org/showthread.php?t=1132821>

UCL

- <http://uclouvain.be/814.html>
- <http://www.uclouvain.be/en-icteam.html>

WIKIPÉDIA

- http://en.wikipedia.org/wiki/Main_Page
- http://fr.wikipedia.org/wiki/Wikip%C3%A9dia:Accueil_principal

Annexes

Voici une liste des fichiers présent sur le CD qui accompagne ce mémoire :

- un dossier *sources/* contenant toutes les sources écrites durant mon stage
- un dossier *submit/* comprenant des exemples de fichiers à soumettre par un instructeur lors de la création d'une soumission
- un fichier *User Manual.pdf* contenant le manuel d'utilisateur de Scavenbox
- un fichier *User Manual.tex* contenant le code LaTeX de ce manuel d'utilisateur
- deux fichiers *Mémoire* reprenant ce mémoire en formats *PDF* et *ODT*
- un fichier *readMe.txt* qui reprend cette liste

Le dossier *sources/* contient les sources suivantes:

- dans le dossier *apache2/*, le fichier *web_app* contient la configuration Apache2 de l'application web de Scavenbox
- dans le dossier *bug_corrector/*, les fichiers contiennent le code du Google Apps Script utilisé pour corriger le bug sur le Google Drive
- dans le dossier *hooks/*, le fichier *pushhook.sh* contient le code exécuté par le hook sur le push pour déclencher l'exécution des tests
- dans le dossier *new/* se trouvent :
 - les scripts Bash utilisés par l'application web de Scavenbox
 - un dossier *google/* contenant l'application qui exporte un Google Apps Script sur le Google Drive
 - un dossier *languages_templates/* contenant les frameworks de tests des différents langages de programmation
 - un dossier *repo_template/* contenant les fichiers communs à tous les repositories qui seront créés
- dans le dossier *vdc/* se trouve l'application Django développée en semaine 3
- dans le dossier *web_app/* se trouve l'application web de Scavenbox, fonctionnant avec le framework Flask
- dans le fichier *sudoers* se trouve les lignes à ajouter au fichier */etc/sudoers* à l'aide de la commande *visudo* lors de l'installation de Scavenbox sur un serveur